

TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

Sistemi di Calcolo Parallelo e Applicazioni

Progetto di fine corso

Andrea Galluzzi 0365026

Alessandro Vassallo 0366572

Introduzione: Obiettivo

Il progetto realizza un nucleo di calcolo parallelo per il *prodotto tra una matrice densa e un multivettore:*

$$Y \leftarrow AX, \qquad A \in \mathbb{R}^{M \times N}, \quad X \in \mathbb{R}^{N \times k}, \quad Y \in \mathbb{R}^{M \times k}.$$

La parallelizzazione è organizzata su due livelli:

- **MPI**
- **CUDA**

Il progetto ha come **obiettivo** il **confronto prestazionale** delle diverse strategie implementate, tramite diverse *campagne di sperimentazione*.

Introduzione: Il nucleo di calcolo

Elemento per elemento il risultato è definito da:

$$Y[i][c] = \sum_{j=0}^{N-1} A[i][j]\, X[j][c], \qquad 0 \le i < M, \quad 0 \le c < k.$$

Per ogni elemento Y[i][c] devono essere combinati N termini del tipo:

$$A[i][j]\, X[j][c].$$

Poiché Y contiene Mk elementi, il numero complessivo di operazioni:

$$\mathrm{FLOP}(M, N, k) = 2MNk.$$



TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

MPI

MPI: Parallelizzazione

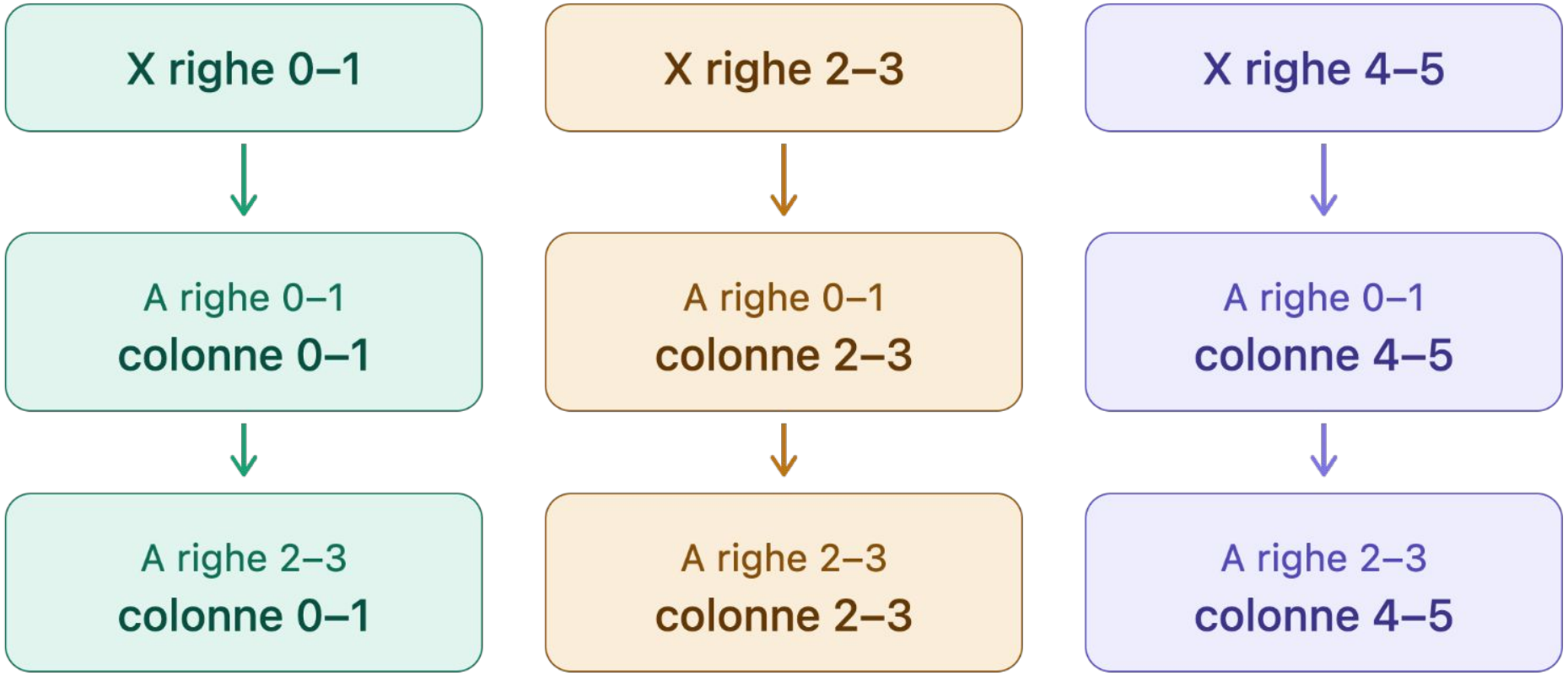
MPI *(Message Passing Interface)* è uno standard per programmare macchine a **memoria distribuita**.

Il **modello di esecuzione** → lo stesso programma viene avviato in P processi, con il proprio spazio di indirizzi **privato**.

Parallelizzazione del nucleo → tre **quesiti** fondamentali:

- *Come si distribuire le matrici A, X e Y tra i processi?*
- *Quale parte del risultato ciascun processo riesce a calcolare con i soli dati che possiede?*
- *Come effettuare la comunicazione di questi dati? In quale direzione deve viaggiare il risultato locale?*

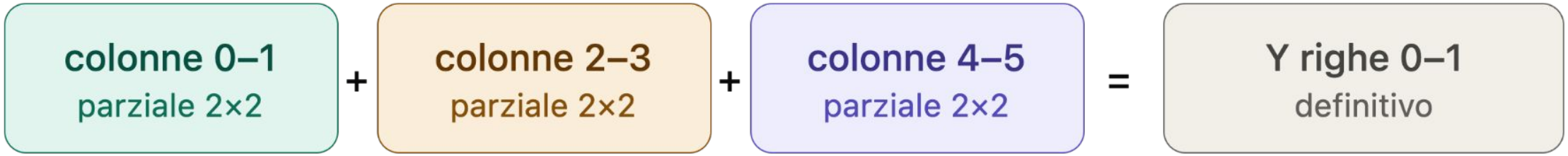
MPI: Il taglio bidimensionale di A



Chi possiede le stesse colonne di A ha bisogno della stessa fetta di X, qualunque siano le sue righe

MPI: Il taglio bidimensionale di A

Contributi alle righe 0–1 di Y, per gruppo di colonne di A



Nessun contributo da solo contiene il risultato: la somma avviene elemento per elemento

MPI: Griglia bidimensionale di processi

Se il taglio del dato è bidimensionale, conviene che lo sia anche il modo in cui si identificano i processi.

MPI offre questa struttura nativamente, sotto il nome di **topologia virtuale cartesiana**.

I P processi vengono organizzati in una **griglia cartesiana** di dimensioni Pr × Pc = P

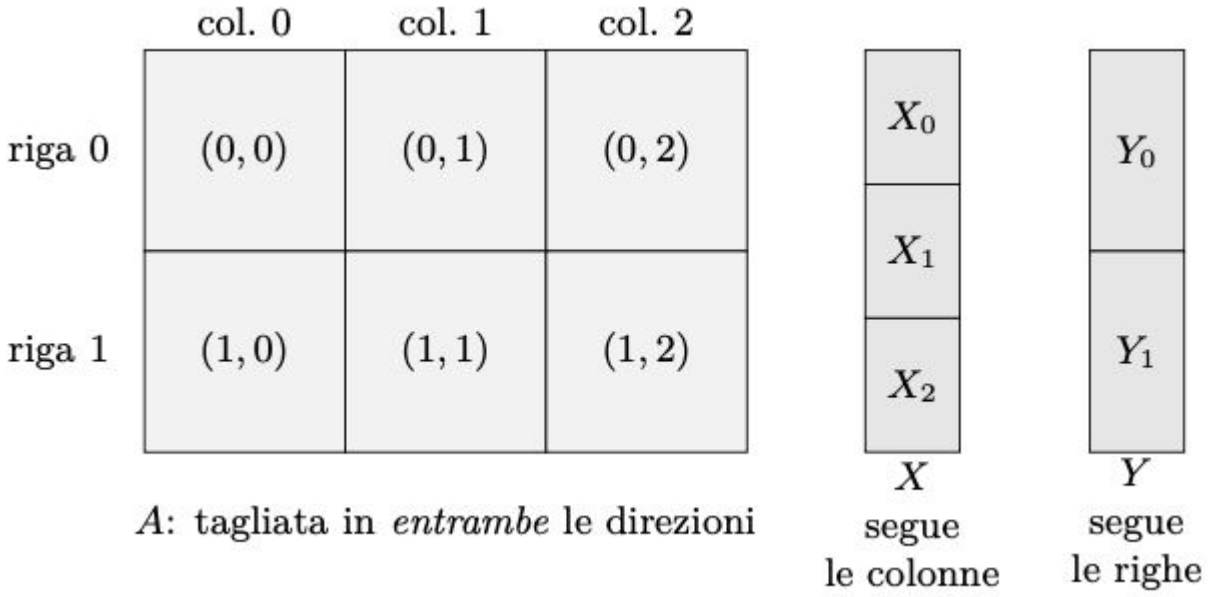
→ `MPI_Cart_create`

La chiamata restituisce un nuovo comunicatore nel quale ogni processo ha, oltre al proprio rank, una coppia di coordinate recuperabile con `MPI_Cart_coords`.

MPI: Griglia e Decomposizione

Decomposizione dei dati

- Ogni processo possiede un rettangolo di A, individuato dalle sue coordinate nella griglia.
- Il multivettore X è tagliato per bande di righe, una per ciascuna banda di colonne di A
- Y per bande di righe, una per ciascuna banda di righe di A. Nessuna delle due è mai tagliata lungo k.



MPI: Sottocomunicatori

La griglia da sola non basta: le collettive vanno eseguite lungo una riga o lungo una colonna, non sull'insieme di tutti i processi. Si ha bisogno di **sottocumunicatori.**

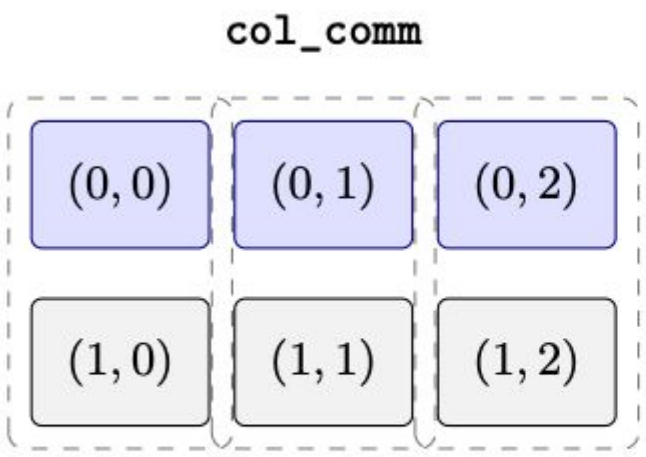
→ `MPI_Cart_sub`: si indica quali dimensioni lasciar variare e quali tenere fisse.

- Fissando la coordinata di riga e lasciando variare quella di colonna si ottiene `row_comm`, che raccoglie i processi di una stessa riga
- Fissando la colonna si ottiene `col_comm`, che raccoglie quelli di una stessa colonna.

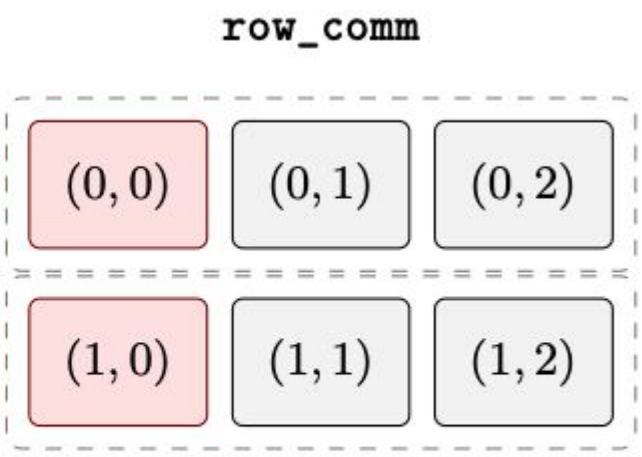
Ogni processo appartiene esattamente a un `row_comm` e a un `col_comm`, e sono i due assi lungo cui viaggiano le collettive dell'algoritmo.

MPI: Sottocomunicatori - Scelta del root

- Il broadcast di X deve partire dal processo di riga 0, che è l'unico a possedere inizialmente quella fetta
- La reduce di Y deve arrivare al processo di colonna 0, che è il proprietario designato di quella porzione del risultato.

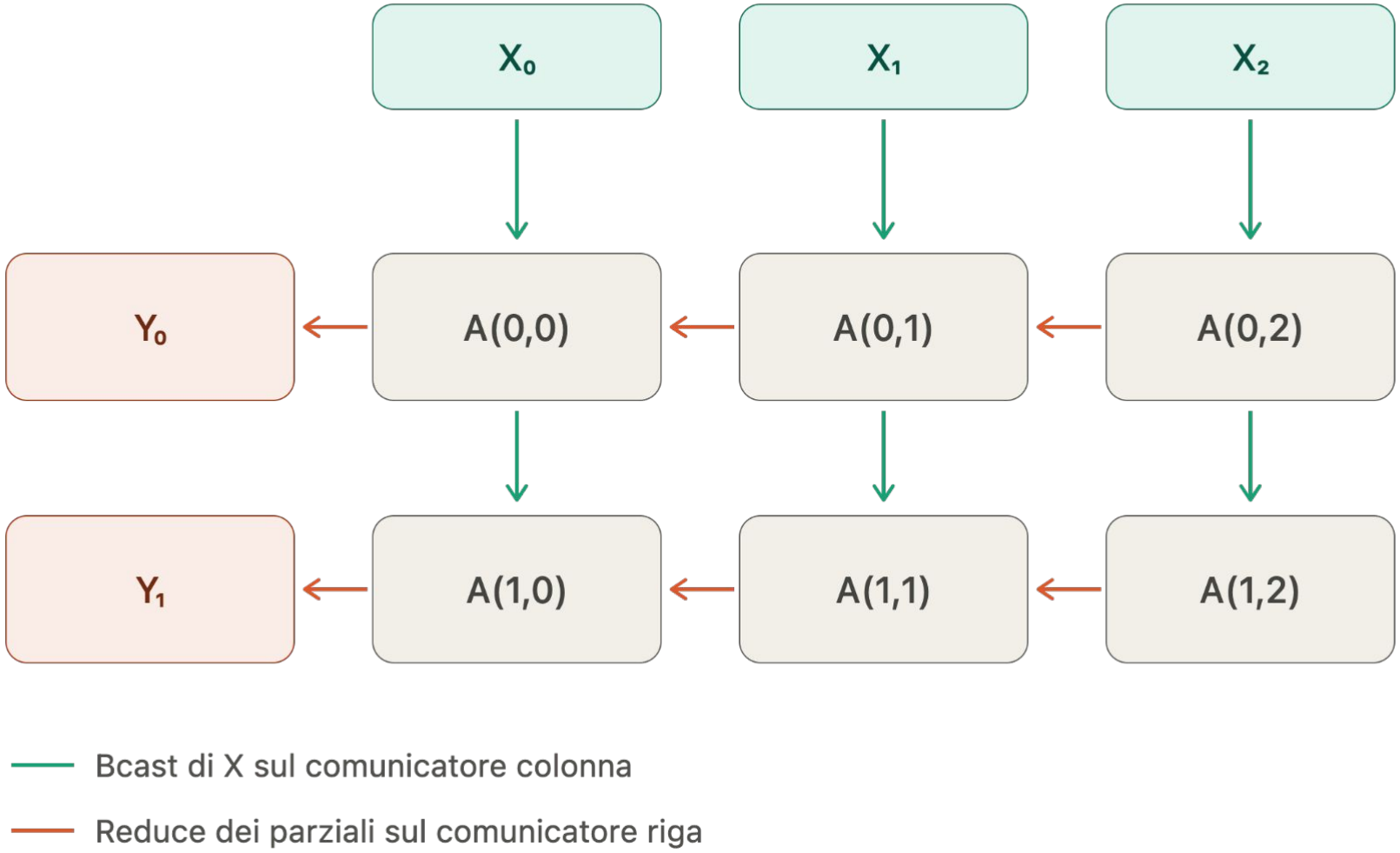


in evidenza la riga 0:
è il root del broadcast di X



in evidenza la colonna 0:
è il root della reduce di Y

MPI: Sottocomunicatori



MPI: Partizionamento degli indici

Posti: $q = \lfloor n/p \rfloor$ e $r = n \bmod p$

con n *numero di elementi da suddividere* e p *numero di processi*, si ha che:

- La funzione che indica ***quanti*** elementi appartengono alla partizione i è

$$\text{size}(n, p, i) = q + \begin{cases} 1 & i < r \\ 0 & \text{altrimenti} \end{cases}$$

- La funzione che indica ***da dove comincia*** la partizione i è

$$\text{start}(n, p, i) = i\,q + \min(i,\ r).$$

MPI: Partizionamento degli indici

Quindi, se si applica il partizionamento lungo le due dimensioni della matrice, ciascun processo calcola:

$$m_{loc} = size(M, P_r, my_row) \qquad n_{loc} = size(N, P_c, my_col)$$

$$row_0 = start(M, P_r, my_row) \qquad col_0 = start(N, P_c, my_col)$$

Ottenendo così la forma dei tre blocchi locali che ogni processo possiede:

$$A_{\text{loc}} : m_{\text{loc}} \times n_{\text{loc}}, \qquad X_{\text{loc}} : n_{\text{loc}} \times k, \qquad Y_{\text{loc}} : m_{\text{loc}} \times k,$$

MPI: L'algoritmo distribuito

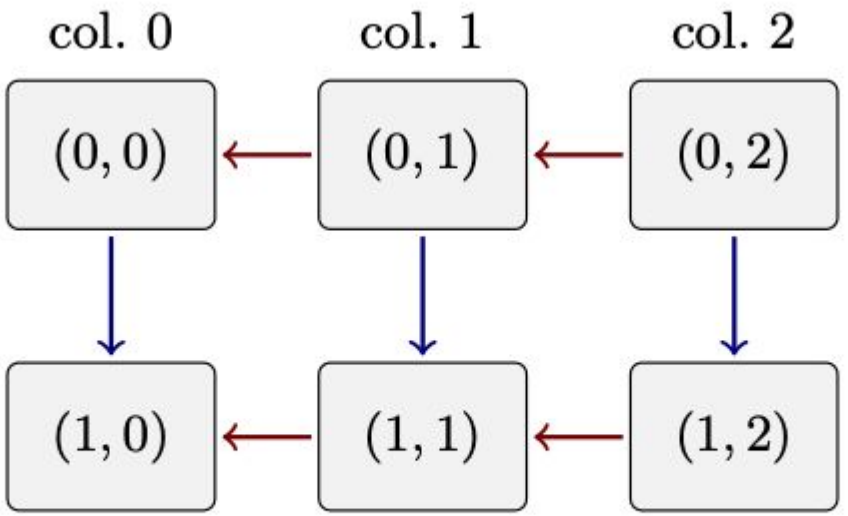
```
MPI_Bcast(X_loc, n_loc*k, SCALAR_MPI_TYPE, 0, grid->col_comm);
```

```
local_gemm(local_gemm_context, X_loc, layout->ldx, Y_loc_part,
layout->ldy);
```

```
MPI_Reduce(Y_loc_part, Y_row_col0, m_loc*k, SCALAR_MPI_TYPE,
MPI_SUM, 0, grid->row_comm);
```

Per mantenere il codice indipendente dalla precisione, `SCALAR_MPI_TYPE` è mappato o su `MPI_FLOAT` o `MPI_DOUBLE` a seconda di come il codice è compilato.

MPI: L'algoritmo distribuito



1. broadcast di X lungo `col_comm`

3. reduce di Y lungo `row_comm`

2. prodotto locale: ogni processo ottiene un contributo *parziale*

Generazione delle matrici

I dati vengono costruiti mediante la libreria `SplitMix64`, utilizzata come generatore pseudocasuale.

Non si tratta di una banale sequenza di numeri pseudocasuali, ma la generazione dipende da tre parametri in input:

$$A[i][j] = g(\text{seed}, \Sigma_A, iN + j)\,, \qquad X[j][c] = g(\text{seed}, \Sigma_X, jk + c)\,.$$

La generazione avviene inizialmente in `double` e la conversione in `scalar_t` avviene solo alla fine.

Generazione delle matrici

Per quanto riguarda la matrice A:

Il rank 0 della griglia materializza A completa e la distribuisce.

È il caso interessante dal punto di vista MPI: i sottoblocchi 2D non sono contigui nella matrice A che è salvata in memoria secondo **row-major**.



Per quanto riguarda la matrice X:

Il rank 0 della griglia materializza la X compatta N × k e con un `MPI_Scatterv` sul `row_comm` della riga 0 della stessa, ne distribuisce i blocchi di righe:

entrambi i lati sono contigui, quindi non serve nessun datatype derivato.

MPI_Type_vector

`MPI_Type_vector`

Definendo sul mittente:

- Quanti blocchi descrivere → `mloc` blocchi
- Quanti elementi consecutivi ciascun blocco → `nloc` elementi
- Lo distanza, misurata in numero di elementi rispetto al “vecchio tipo”, `SCALAR_MPI_TYPE` → `N` elementi (lunghezza di una riga)

Restituisce un **nuovo tipo da inviare.**

Per il processo (r,c) il blocco da inviare è quindi:

$$A[row_0 : row_0 + m_{loc}, col_0 : col_0 + n_{loc}].$$

MPI_Scatterv

`MPI_Scatterv`

L'operazione di Scatter prende un insieme di dati, lo suddivide e lo distribuisce agli altri processi.

La versione "variabile" ha due array in più, significativi solo sul root:

- `sendcounts[i]` → quanti elementi al processo i (possono essere tutti diversi);

- `displs[i]` → a che offset del `sendbuf` comincia la fetta di i, misurato in unità di sendtype.



Backend CUDA

Backend CUDA: panoramica

Kernel CUDA sviluppati:

- CUDA Naive → un thread per ogni elemento di Y
- CUDA Warp → un warp per riga di Y
  - variante X row-major
  - variante X column-major
- CUDA Warp + Shared Memory → staging di X in shared memory

Riferimento esterno:

- cuBLAS → GEMM ottimizzata NVIDIA

Struttura comune dei backend CUDA

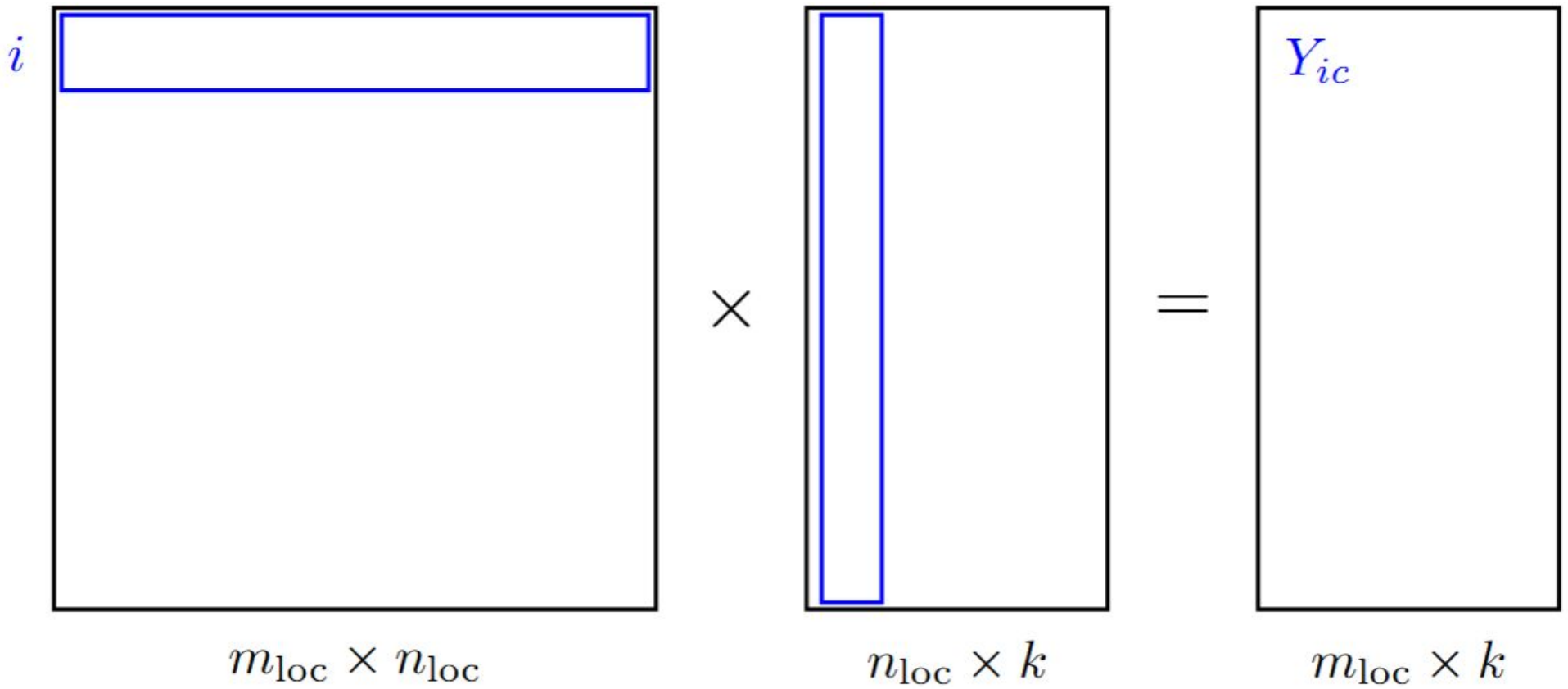
Ogni backend CUDA segue queste tre fasi:

1. `local_gemm_create()` → *preprocessing una volta prima delle repetition*
   - inizializzazione del contesto CUDA
   - allocazione buffer su GPU
   - creazione CUDA events
   - **`copia di A_loc`**: `Host → Device` (A rimane residente nel Device)
2. `local_gemm()` → *eseguita a ogni repetition*
   - **copia X_loc**: host → Device
   - esecuzione del kernel CUDA / chiamata cuBLAS
   - **copia Y_loc_part**: Device → Host
3. `local_gemm_destroy()`
   - rilascio buffer
   - distruzione eventi CUDA
   - rilascio del contesto

CUDA Naive

A partire da identificativo globale del thread ricaviamo $\boxed{\text{Thread } t \rightarrow Y_{ic}}$

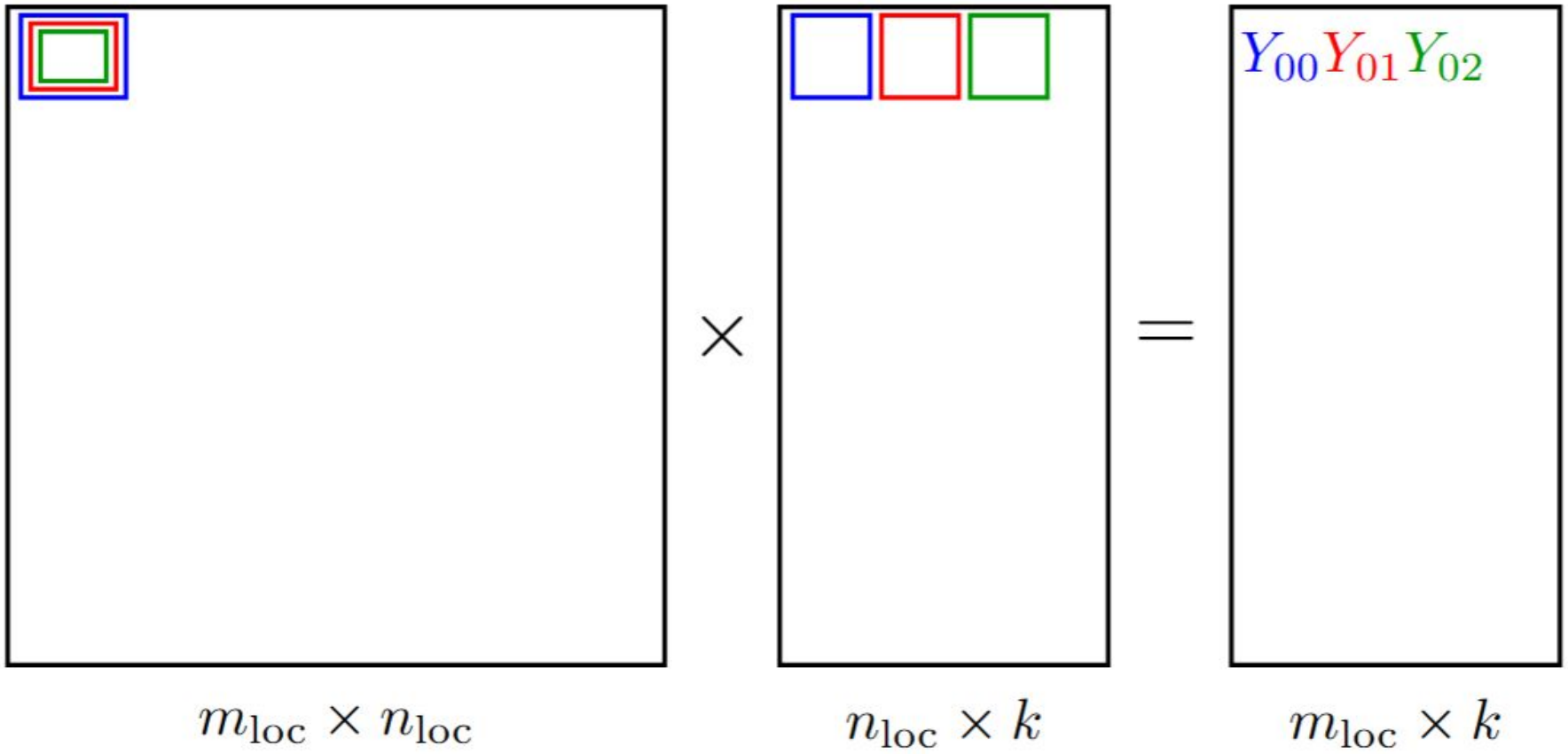
Ogni thread mantiene **un accumulatore** e percorre tutta la dimensione j.



$$Y_{ic} = \sum_{j=0}^{n_{\text{loc}}-1} A_{ij} X_{jc}$$

CUDA Naive: limite del riuso di A

Per una stessa riga i e una stessa iterazione j, più thread richiedono lo stesso Aij, ma ciascun thread lo utilizza solo per il proprio elemento di Y.



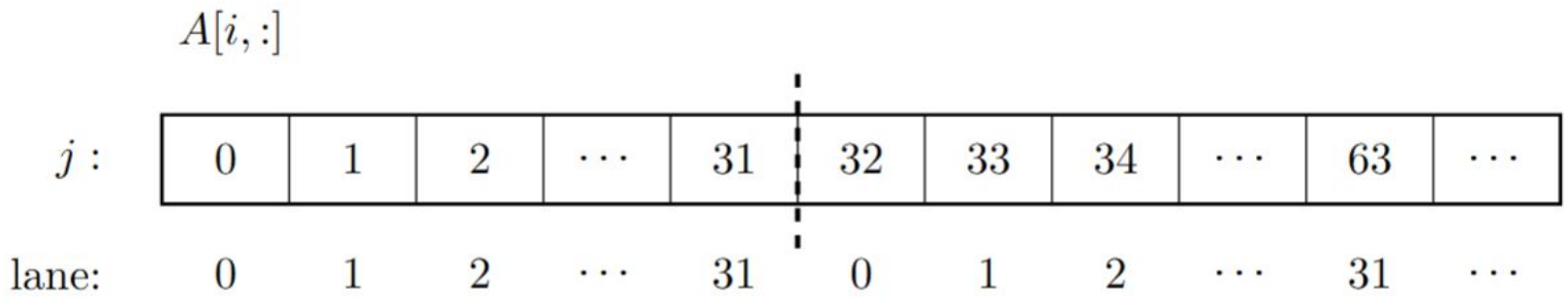
Nessun riuso esplicito di Aij all'interno del thread.

CUDA Warp: un warp per riga di Y

Un intero warp viene assegnato a una riga di Y $\boxed{1 \text{ warp} \longrightarrow 1 \text{ riga di } Y}$

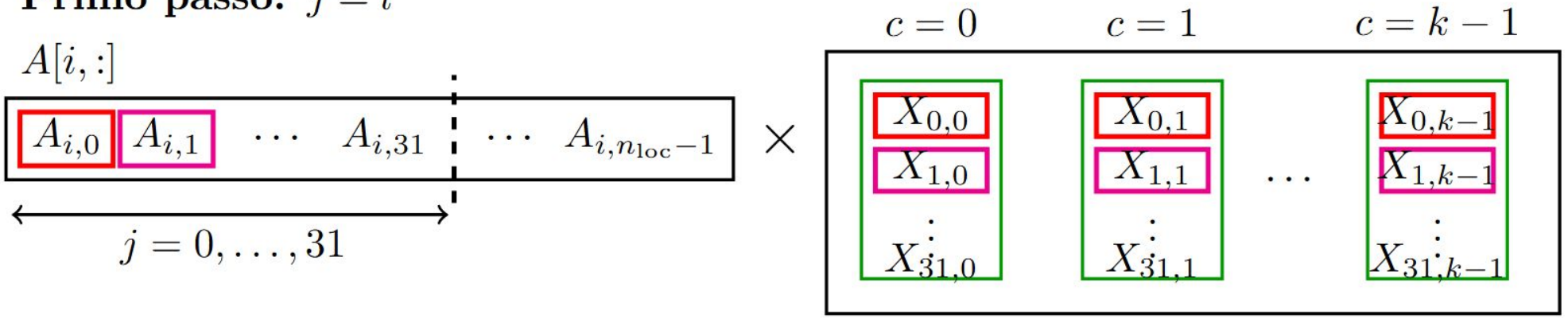
- Le 32 lane si dividono la dimensione j. La lane l visita: $\boxed{\text{lane } l: \quad j = l,\ l+32,\ l+64, \ldots}$

- Lane consecutive accedono a elementi consecutivi della stessa riga di A



CUDA Warp: riuso esplicito di A

Primo passo: $j = l$



$$a = A_{i,j}$$

$$\text{acc}_l[c] \mathrel{+}= a\, X_{j,c}, \qquad 0 \le c < k$$

Accumulatori al primo passo:

lane 0 $a = A_{i,0}$

$\text{acc}_0[0] \mathrel{+}= aX_{0,0}$

$\text{acc}_0[1] \mathrel{+}= aX_{0,1}$

$\vdots$

$\text{acc}_0[k-1] \mathrel{+}= aX_{0,k-1}$

lane 1 $a = A_{i,1}$

$\text{acc}_1[0] \mathrel{+}= aX_{1,0}$

$\text{acc}_1[1] \mathrel{+}= aX_{1,1}$

$\vdots$

$\text{acc}_1[k-1] \mathrel{+}= aX_{1,k-1}$

lane 31 $a = A_{i,31}$

$\text{acc}_{31}[0] \mathrel{+}= aX_{31,0}$

$\vdots$

$\text{acc}_{31}[k-1] \mathrel{+}= aX_{31,k-1}$

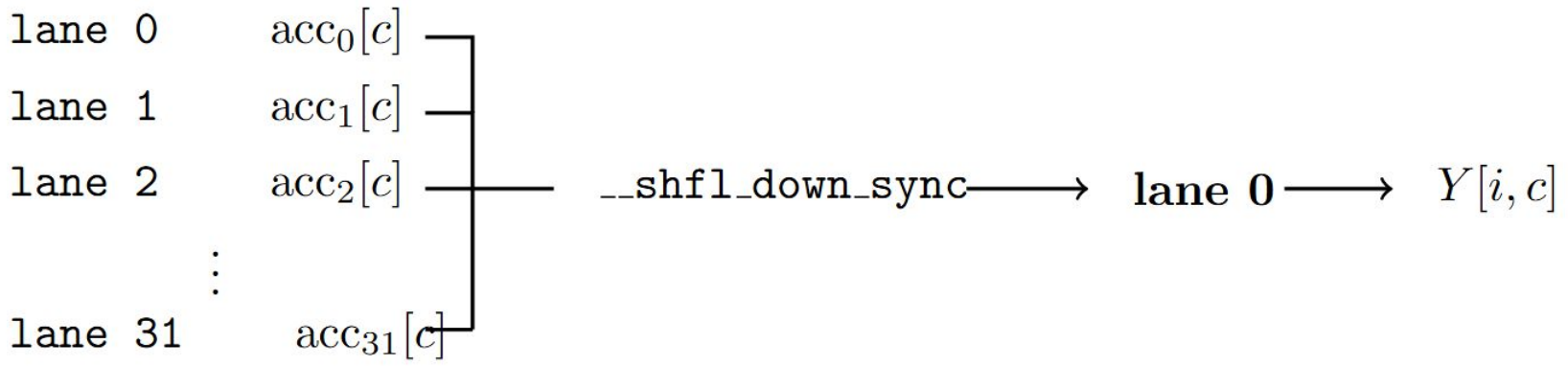
Un caricamento di A per eseguire k aggiornamenti!

CUDA Warp: riduzione intra-warp

Al termine del ciclo su j, ogni lane possiede solo un contributo parziale alla riga di Y

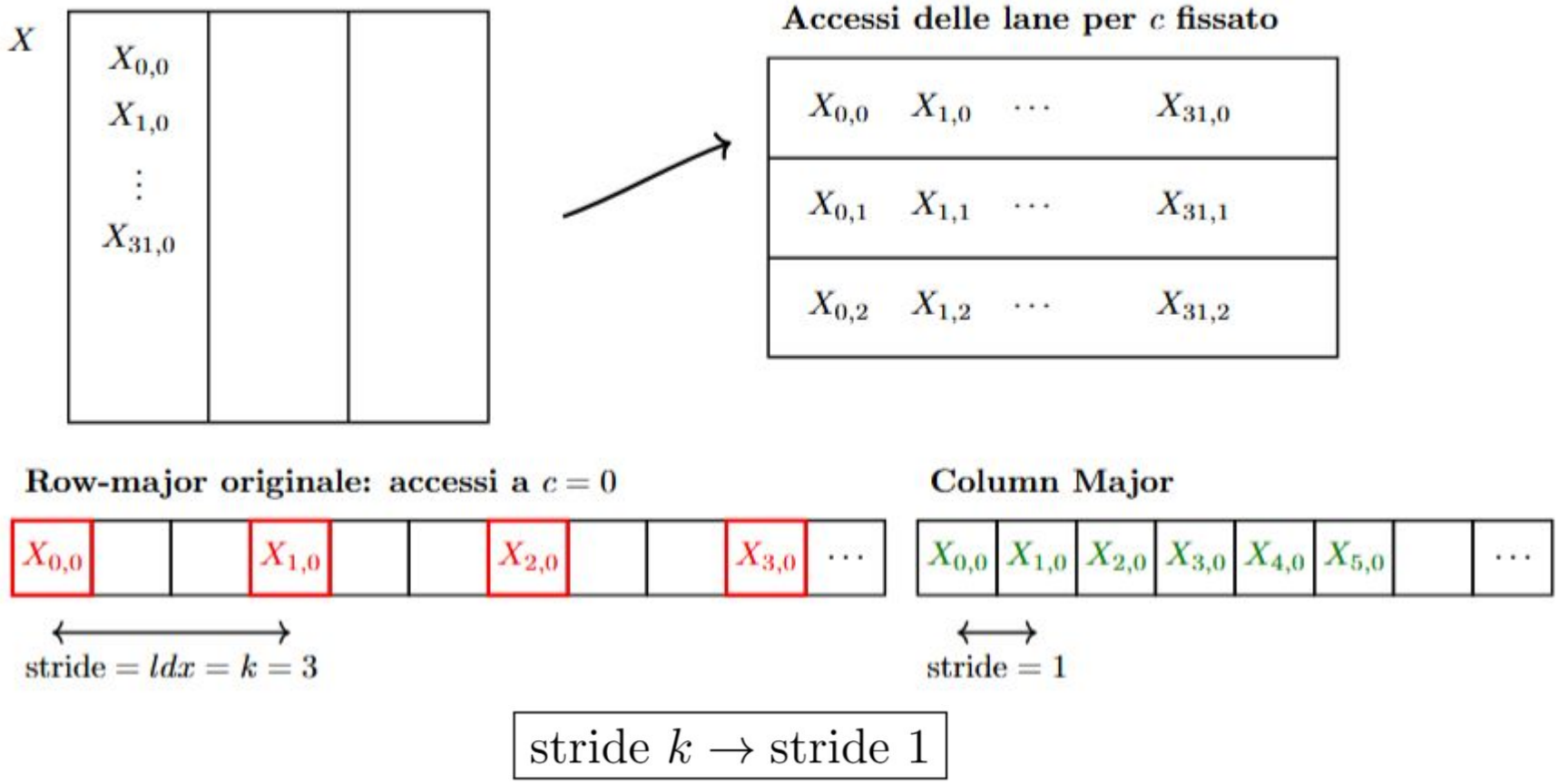
$$Y_{ic} = \sum_{l=0}^{31} \mathrm{acc}_l[c]$$

- i contributi vengono ridotti tramite `__shfl_down_sync`
- al termine lane 0 possiede i k valori completi
- lane 0 scrive la riga in Y



CUDA Warp: accessi a X

La distribuzione di j tra le lane rende favorevoli gli accessi ad A, ma introduce accessi strided a X in layout row-major. Esempio k=3



stride $k \rightarrow$ stride 1

CUDA Warp + Shared Memory: staging X

Manteniamo la mappatura **warp-per-row** di cuda_warp, ma cambiamo il percorso di X.

Infatti:

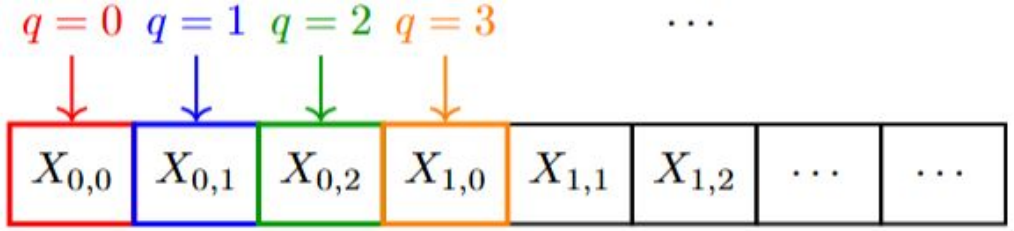
- X resta Row-Major
- un tile di X viene caricato in shared memory
- i warp leggono X dal tile invece che dalla global memory

CUDA Warp + Shared Memory: load

Caricamento cooperativo di X: il tile non viene caricato dai singoli warp ma da tutti i thread del **blocco**. Ogni thread del blocco percorre gli elementi del tile mediante un indice lineare **q**, inizializzato a `threadIdx.x`.

$$r = \left\lfloor \frac{q}{k} \right\rfloor, \qquad c = q \bmod k$$

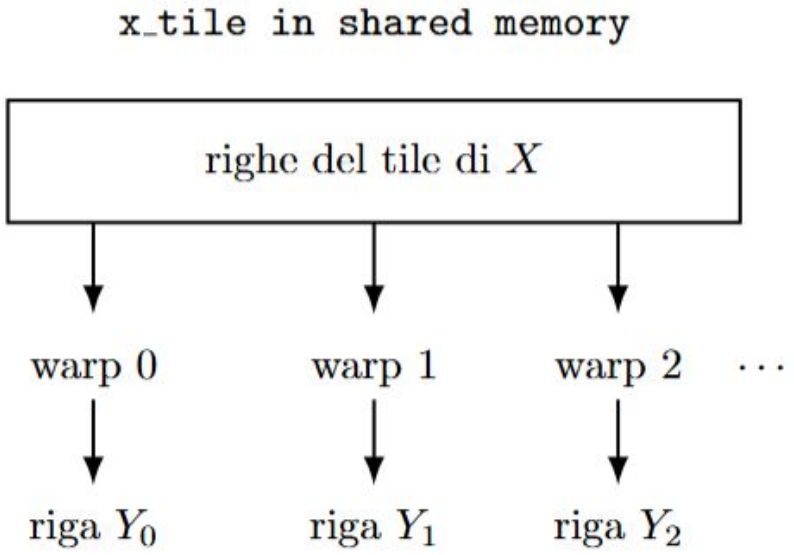
I thread consecutivi leggono quindi elementi consecutivi della rappresentazione row-major.



Dopo il caricamento `__syncthreads()` sincronizza il blocco e garantisce che il tile sia completamente disponibile prima dell'utilizzo da parte dei warp.

CUDA Warp + Shared Memory: riuso

Una volta caricato in shared memory, lo stesso tile di X, viene utilizzato da tutti i warp del blocco



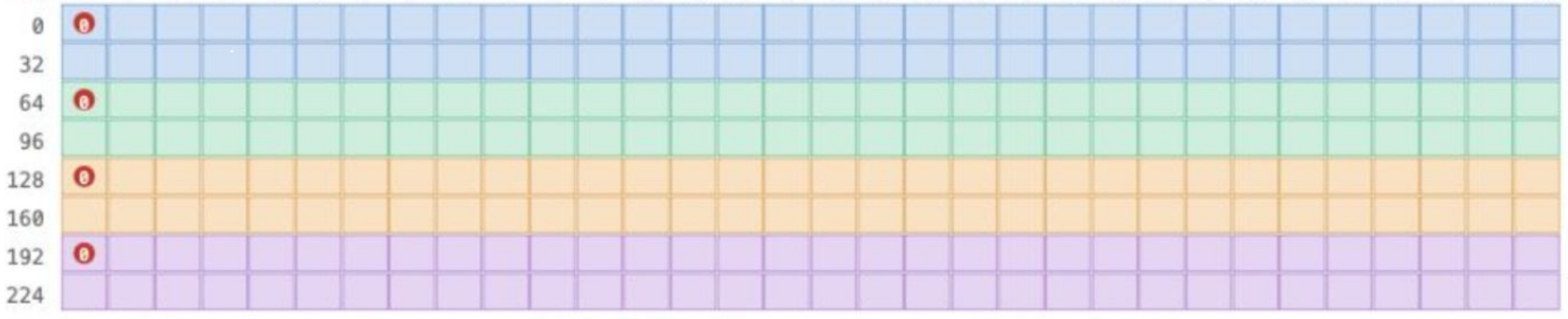
I warp del blocco lavorano su **righe differenti di A e Y**, ma utilizzano lo **stesso X**.

CUDA Warp + Shared Memory: padding

La shared memory è organizzata in **32 bank**. Se lane diverse accedono a indirizzi diversi dello **stesso bank**, gli accessi vengono parzialmente serializzati.

$$X_{\text{tile}}\,[l \cdot \texttt{tile_row_stride} + c] \quad \text{con} \quad \texttt{tile_row_stride} = k + \texttt{SMEM_PAD}$$

- Caso **senza padding** con k=32 → in FP64 ogni elemento occupa 2 word da 32 bit. Lo stride tra gli accessi di lane consecutive è: $32 \cdot 2 = 64 \text{ word}, \qquad 64 \bmod 32 = 0$
  - Di conseguenza gli accessi delle lane ricadono periodicamente sugli stessi bank→ CONFLITTO di BANCHI

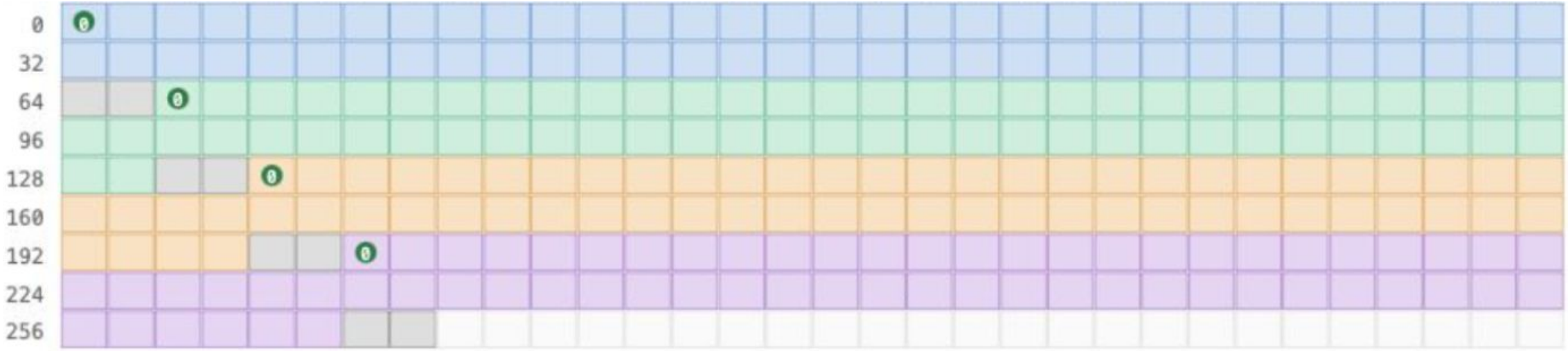


CUDA Warp + Shared Memory: padding

La shared memory è organizzata in **32 bank**. Se lane diverse accedono a indirizzi diversi dello stesso bank, gli accessi vengono parzialmente serializzati.

$$X_{\text{tile}}\,[l \cdot \texttt{tile_row_stride} + c] \quad \text{con} \quad \texttt{tile_row_stride} = k + \texttt{SMEM_PAD}$$

- Caso **con padding=1** con k=32 $\rightarrow 33 \cdot 2 = 66 \text{ word}, \qquad 66 \bmod 32 = 2$
  - Gli accessi delle lane non ricadono più sempre sugli stessi bank, ma vengono distribuiti con uno spostamento di 2 bank.



CUDA Warp + Shared Memory: dim. tile

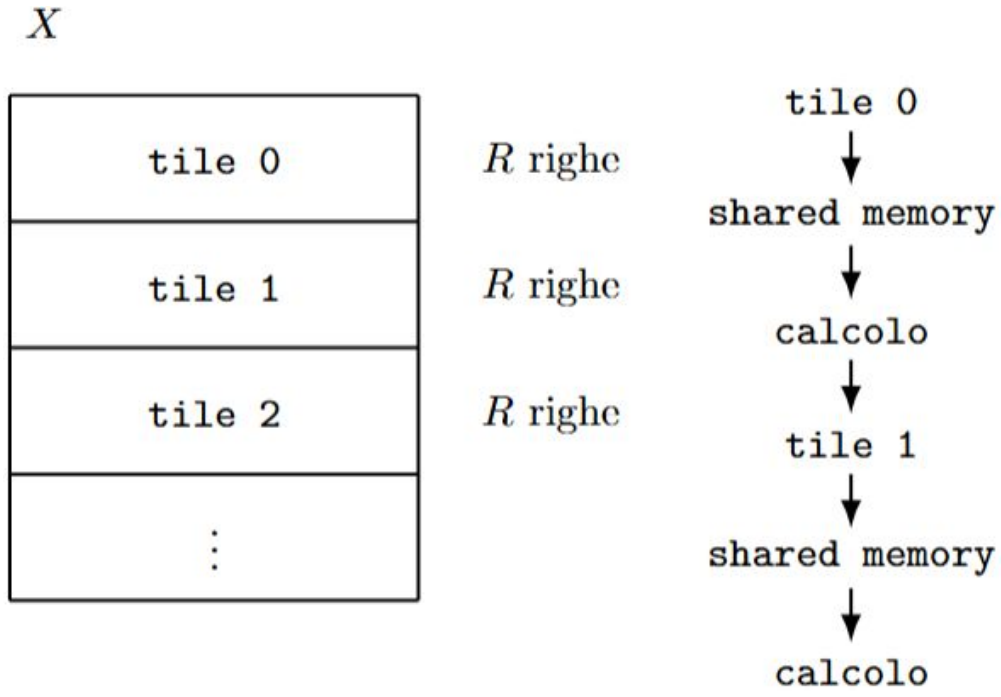
X non viene caricata interamente in shared memory: viene suddivisa in tile di R righe. Quindi concettualmente la shared memory è definita come:

$$\texttt{smem_per_tile} = R\,(k + \texttt{SMEM_PAD})\,\texttt{sizeof}(\texttt{scalar_t})$$

dove R è il numero di righe del tile

La dimensione del tile viene scelta in modo da usare la shared memory disponibile senza ridurre in modo eccessivo l'occupancy.

- tile più grande → meno iterazioni e meno barriere di sincronizzazione ma spesso occupancy più bassa
- tile più piccolo → più margine di occupancy ma più sincronizzazioni e più iterazioni



cuBLAS: riferimento esterno

cuBLAS viene usata come riferimento esterno per confrontare i kernel CUDA sviluppati nel progetto.

- Implementa la stessa interfaccia degli altri backend
- Adattamento row major → column major
  - Il progetto usa row major ma cublas interpreta gli operandi in column major.
  - Di fatto non viene effettuata una conversione fisica ma reinterpretiamo i buffer come trasposti

$$A_{cm} = A_{rm}^T, \qquad X_{cm} = X_{rm}^T, \qquad Y_{cm} = Y_{rm}^T$$

Quindi

$$Y_{cm} = X_{cm} A_{cm}$$

cuBLAS rappresenta una baseline di libreria, non un ulteriore ottimizzazione dei nostri kernel.

Infatti sceglie internamente tiling, distribuzione del lavoro e occupancy; il confronto è quindi puramente sperimentale.

TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

Fase Sperimentale

Fase sperimentale

- Server di calcolo del dipartimento
- GPU compilata per la sua architettura → sm_75
- 5 warm-up + 20 repetition misurate
- seed fisso per la generazione delle matrici → *20252026*

Disegno degli esperimenti:

- **FASE 1** → Tuning dei backend CUDA
  - In questa fase cerchiamo di fissare delle configurazioni che rappresentino la versione del kernel ottimale
- **FASE 2** → Variazione di scala e forma
- **FASE 3** → FP64 vs FP32
- **FASE 4** → MPI + CUDA (multi rank)



TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

FASE 1 - Tuning

FASE 1 - configurazione dei back CUDA

Obiettivo: scegliere una configurazione robusta per ciascun backend prima di cambiare dimensione o forma del problema.

4 campagne caratterizzano questa fase:

- Tuning del block thread
  - CUDA Naive
  - CUDA Warp (Column Major)
- Confronto layout X
  - CUDA Warp
- Tuning BLOCK x Granularità del Tile
  - CUDA Warp Shared Memory
- Influenza del padding
  - CUDA Warp Shared Memory

$$M = N = 16384, \qquad P = 1$$

$$k \in \{3, 6, 8, 20, 32\}$$

FASE 1.1 – Tuning Block Threads

Obiettivo: individuare una configurazione che mantenga prestazioni robuste su tutti i valori di k, non semplicemente quella migliore in un singolo caso.

Lo sweep viene eseguito per:

- CUDA Naive
- CUDA Warp (Column Major)

Vlene effettuato lo sweep dei blocchi: $BLOCK \in \{64, 128, 192, 256, 384, 512, 1024\}$

Tutti i valori sono multipli della dimensione del warp (32 thread)

Per ogni BLOCK vengono testati tutti i valori $k \in \{3, 6, 8, 20, 32\}$

FASE 1.1 – Tuning Block Threads

Criterio di selezione

Per ogni configurazione di (BLOCK, k) si ottiene un risultato $P(B,k) = \mathrm{GFLOPS}_{\mathrm{kernel}}(B,k)$

Per ciascun k viene individuato il blocco che si comporta meglio e quindi per ogni k è presente

$$\max_{B'} P(B',k)$$

Si calcola il rapporto tra $P(B,k)$ e $\max_{B'} P(B',k)$ di ogni configurazione (BLOCK,k).

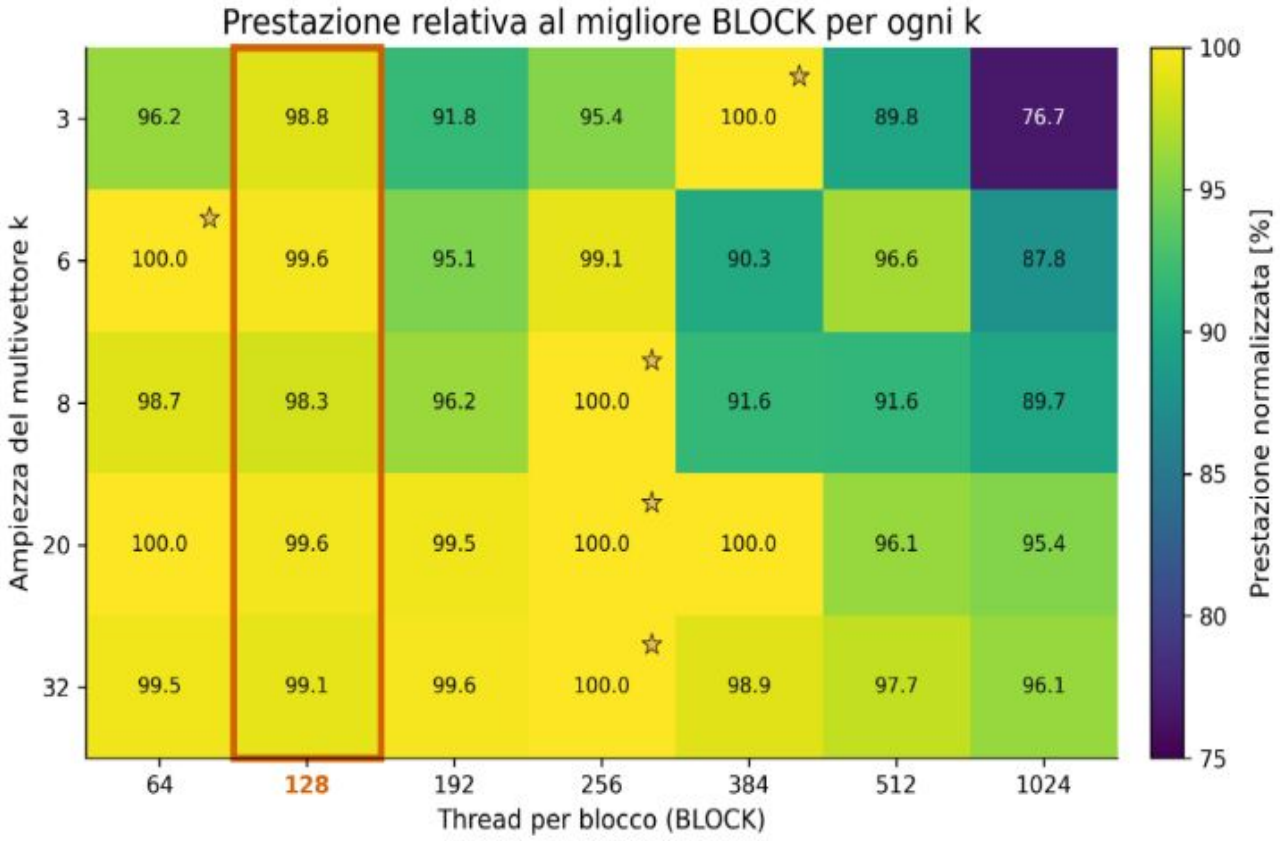
$$R(B,k) = \frac{P(B,k)}{\max_{B'} P(B',k)}$$

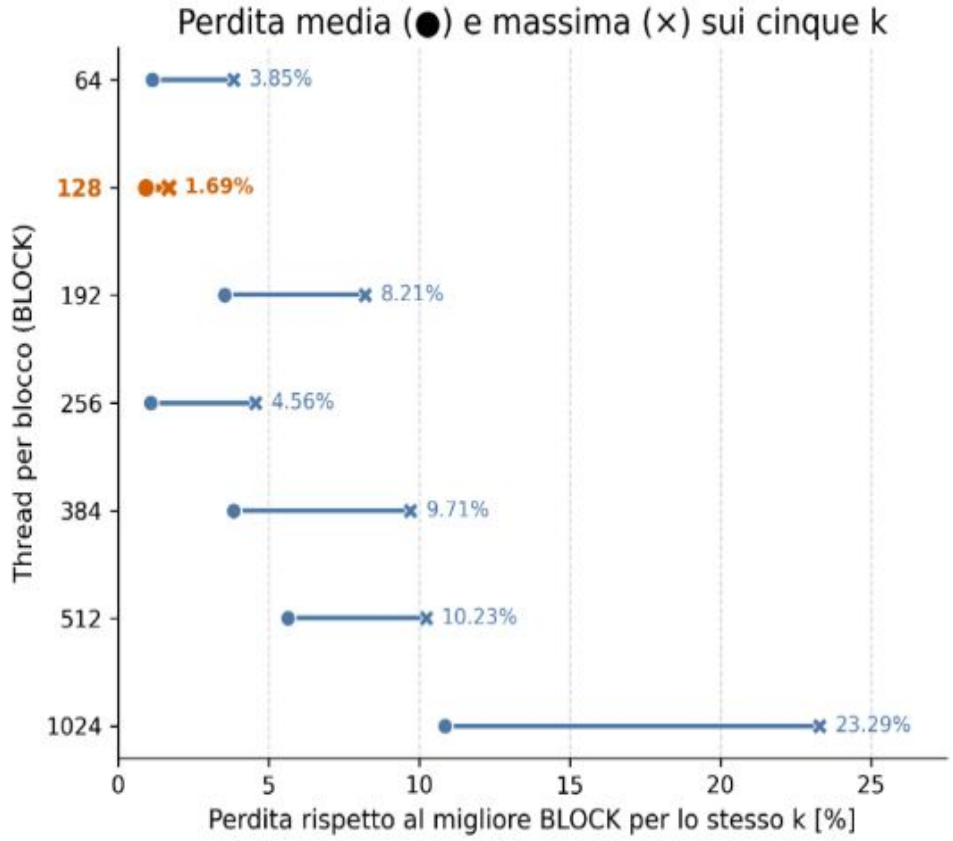
Viene calcolato $L(B,k) = 1 - R(B,k)$ per ogni configurazione (BLOCK, k).

Viene selezionato il blocco che ha tra i suoi k $L(B,k)$ quello che ha tra tutti i casi la minima:

$$\max_{k} L(B,k)$$

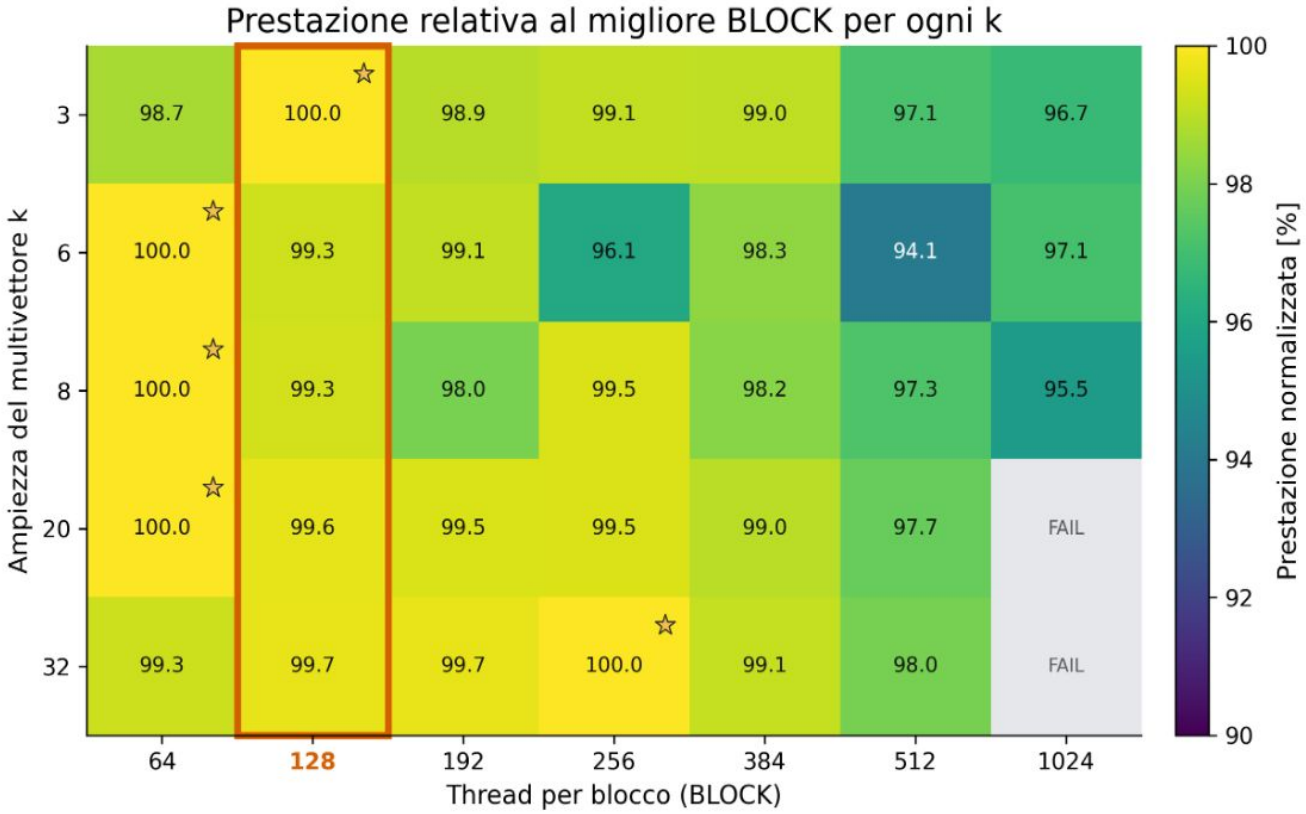
FASE 1.1 - CUDA Naive risultati

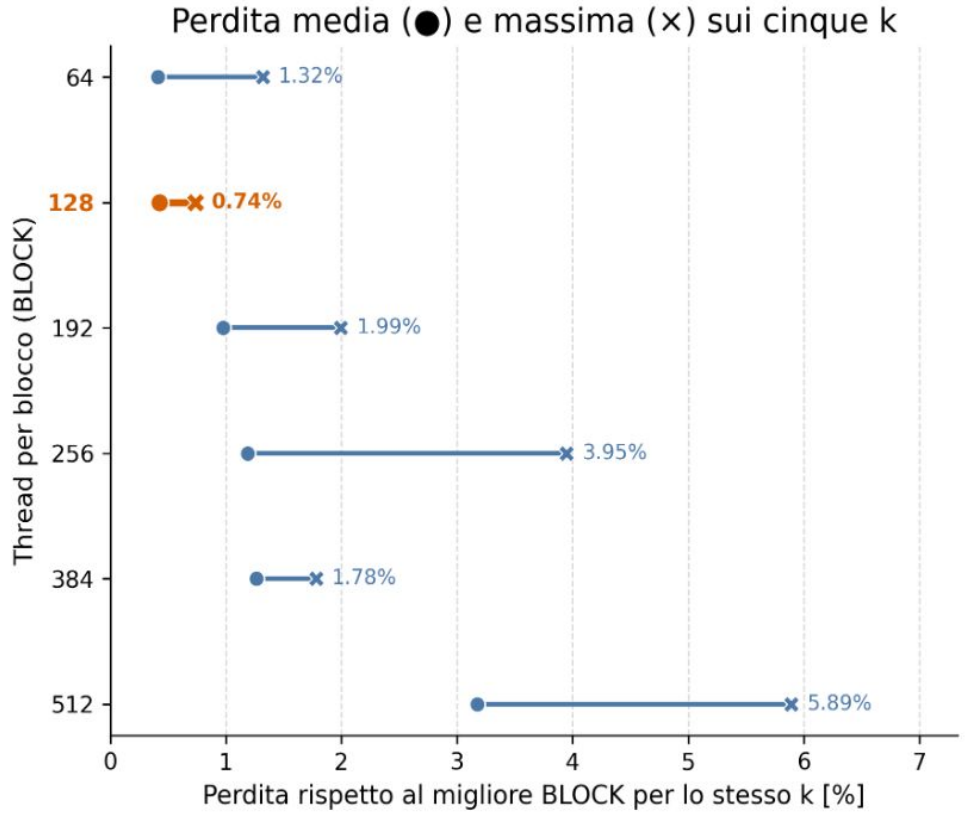




BLOCK=128 quindi non è necessariamente il migliore per ogni singolo k, ma è la configurazione più robusta sull'insieme dei valori di k.

FASE 1.1 - CUDA Warp risultati





BLOCK=128 quindi non è necessariamente il migliore per ogni singolo k, ma è la configurazione più robusta sull'insieme dei valori di k.

FASE 1.2 - Layout di X

Obiettivo: verificare effetto layout di X sugli accessi del kernel CUDA Warp

Condizioni:

$$BLOCK = 128 \qquad k \in \{3, 6, 8, 20, 32\}$$

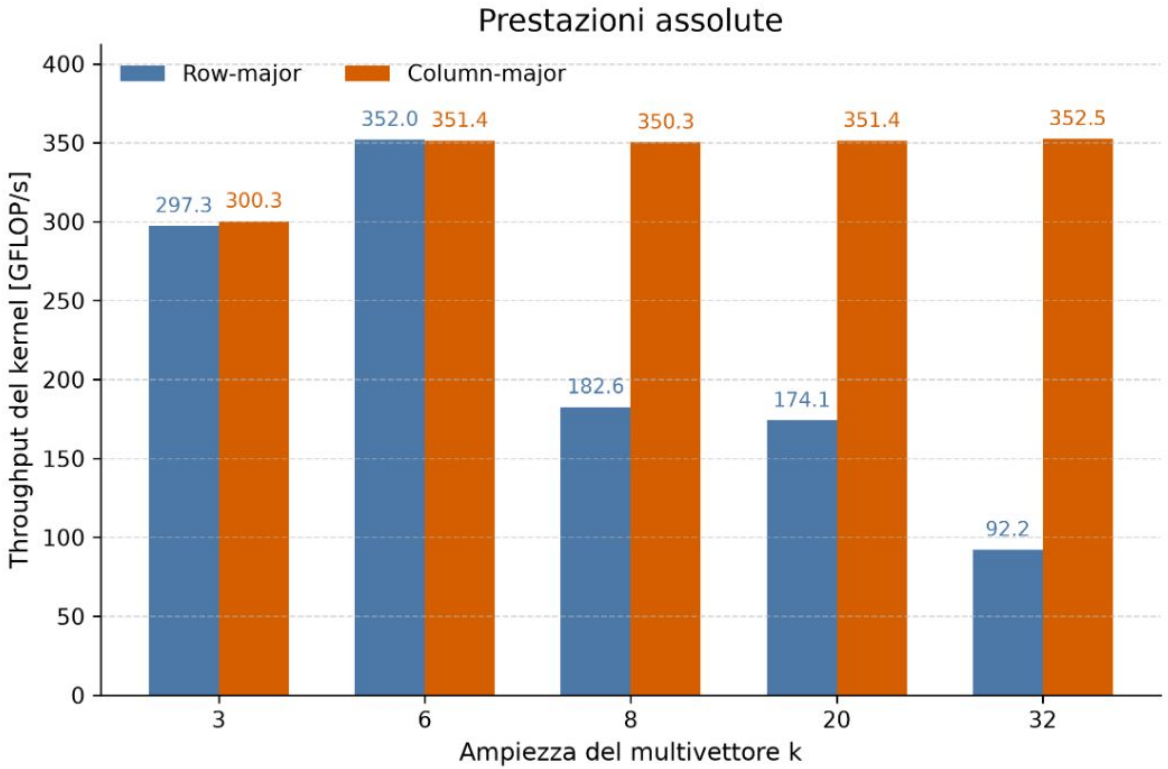
Confronto:

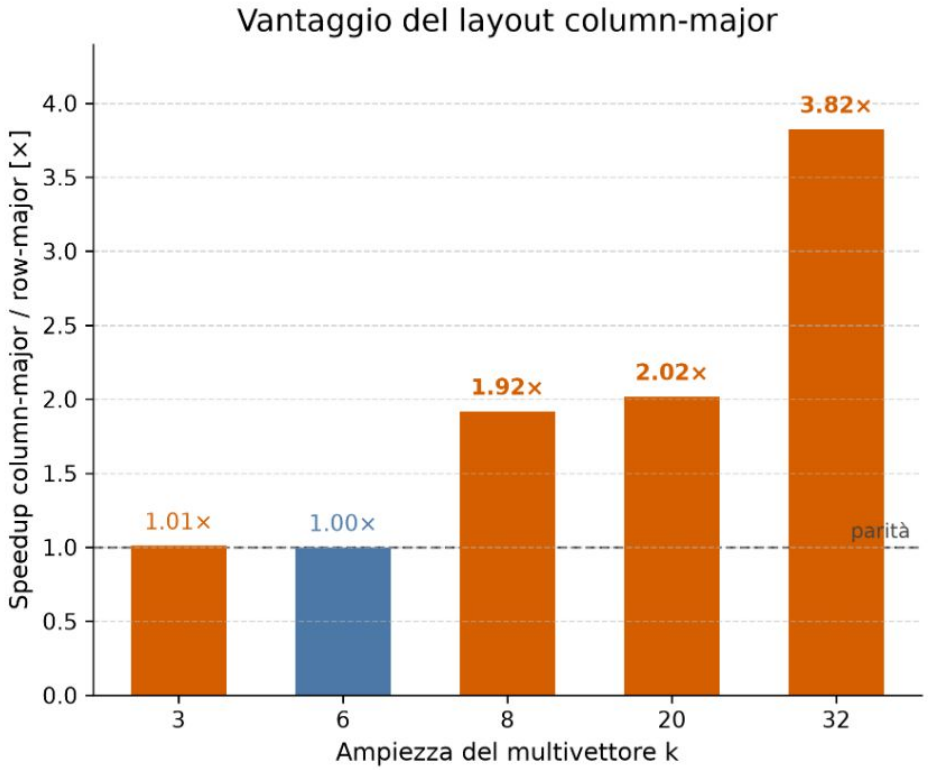
- X row major
- X column major

In row-major, a c fissato, lane consecutive accedono a elementi distanti k;

In column-major gli stessi elementi risultano consecutivi.

FASE 1.2 - Layout di X risultati





FASE 1.3 - Block + Granularità

Obiettivo: scegliere congiuntamente la dimensione del blocco e la granularità usata per determinare il numero di righe del tile

Queste due vengono studiate insieme perché entrambe influenzano la dimensione effettiva del tile e l'uso della shared memory.

Parametri:

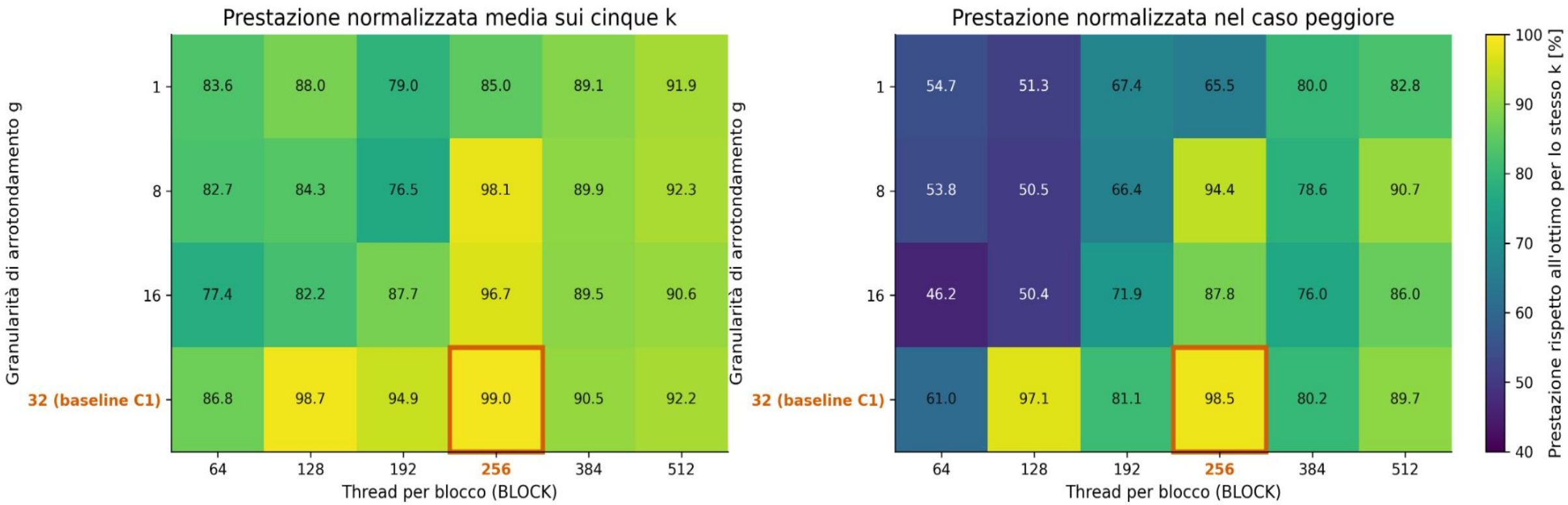
$$BLOCK \in \{64, 128, 192, 256, 384, 512, 1024\}$$

$$g = \text{TILE_GRANULARITY} \in \{1, 8, 16, 32\}$$

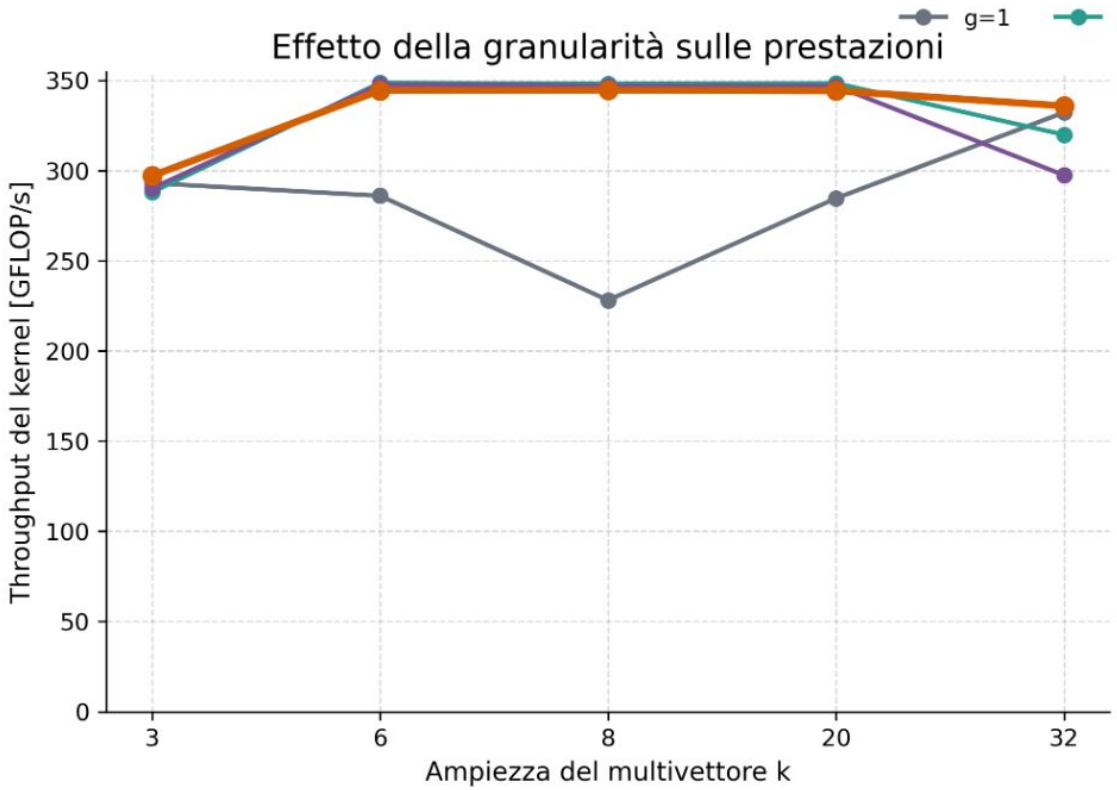
Il criterio è lo stesso della fase 1.1; cambia soltanto lo spazio delle configurazioni.

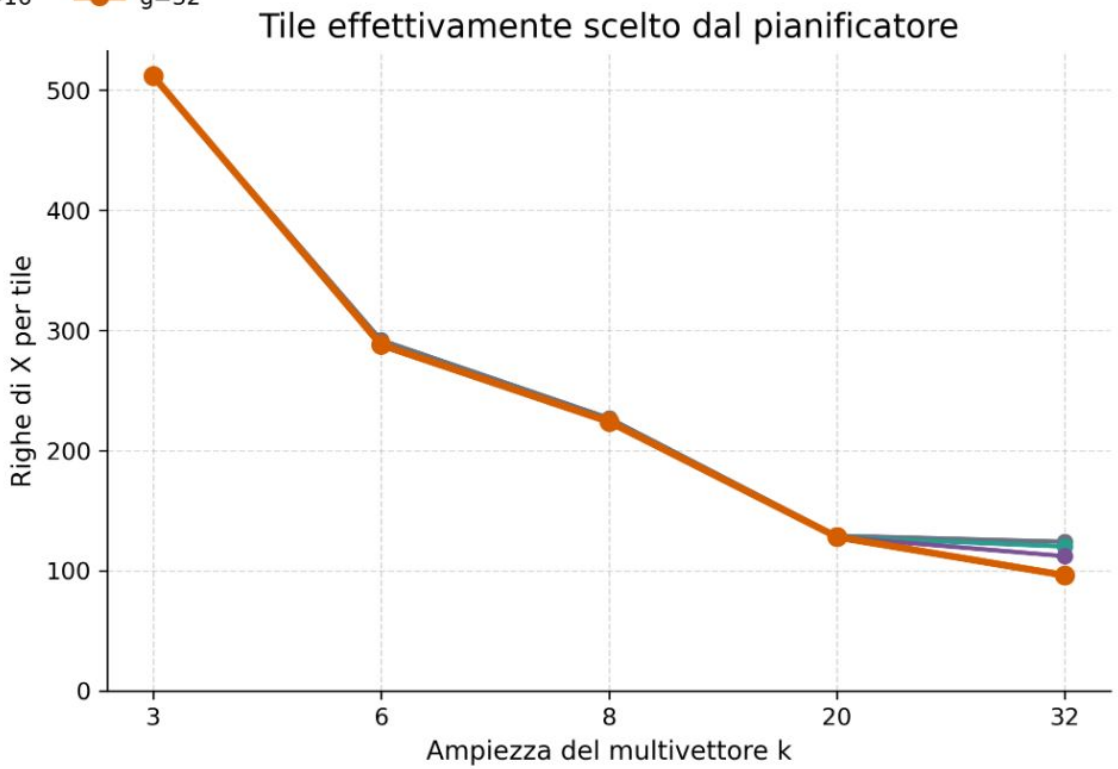
In Fase 1.1 una configurazione è un BLOCK, mentre in questa è una coppia BLOCK-granularità.

FASE 1.3 - Block + Granularità risultati



FASE 1.3 - Block + Granularità risultati





FASE 1.4 - Effetto del padding

Obiettivo: verificare se il padding del tile in shared memory riduce i bank conflict senza penalizzare troppo gli altri casi.

Parametri:

- PADDING=1
- PADDING=0

Configurazione:

$$BLOCK = 256 \qquad g = 32$$

FASE 1.4 - Effetto del padding risultati

k	$PAD = 0$	$PAD = 1$	Δ
3	292.8	296.7	+1.3%
6	315.8	351.8	+11.4%
8	327.5	352.4	+7.6%
20	353.9	352.2	−0.5%
32	177.1	343.7	+94.1%

TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

FASE 2 - Caratterizzazione dimensionale

FASE 2

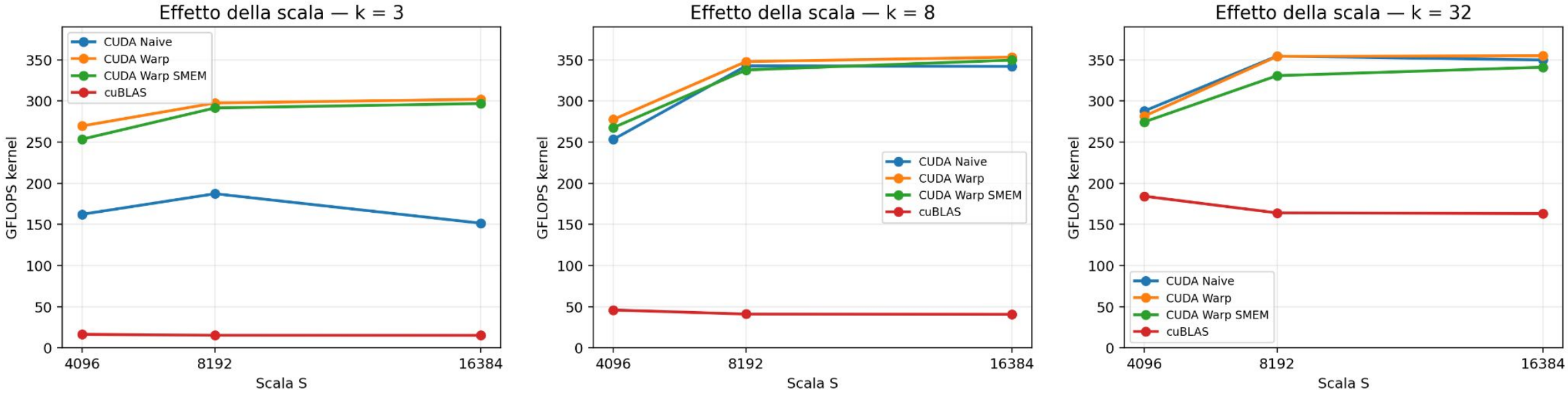
Obiettivo: Verificare se le configurazioni selezionate nella Fase 1 mantengono un comportamento stabile quando cambiano **scala** e **forma** del problema.

- La dimensione della varia tra 4096, 8192 e 16384.
- P=1
- Per ogni scala vengono considerate 3 forme, mantenendo $MN \simeq S^2$ in modo tale da mantenere il lavoro computazionale costante

S	$M = N$	$N = 2M$	$M = 3N$
4096	4096×4096	2880×5760	7104×2368
8192	8192×8192	5760×11520	14208×4736
16384	16384×16384	11520×23040	28416×9472

FASE 2 - Effetto della scala

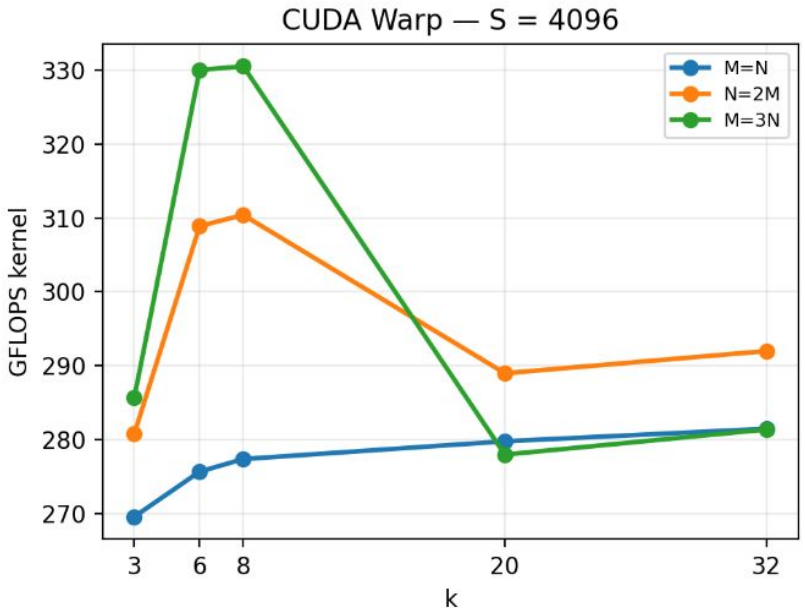
Valutiamo il caso quadrato: M=N=S con $S \in \{4096, 8192, 16384\}$

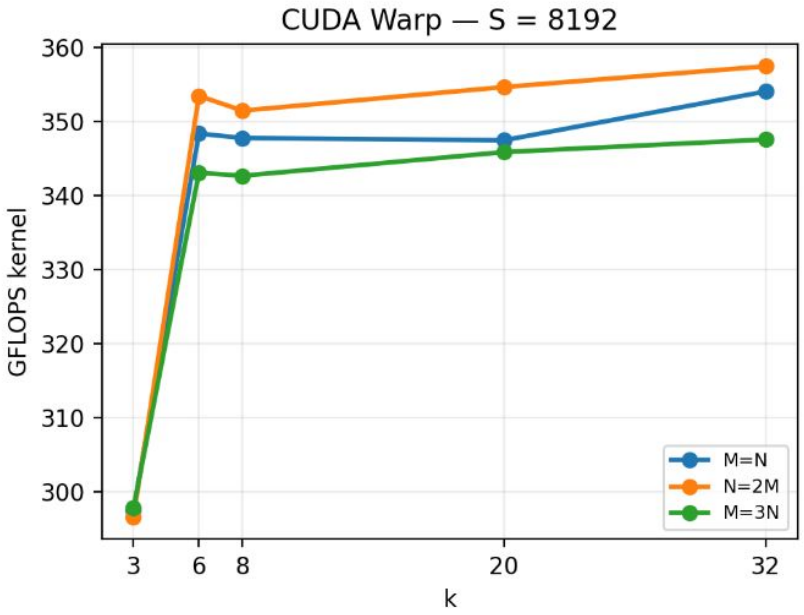


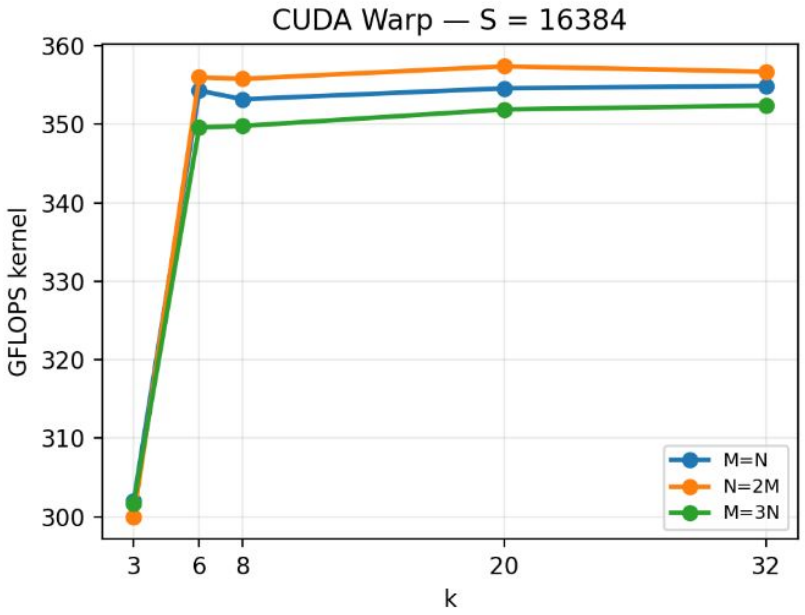
- Notiamo come in generale il kernel che si comporta comunque meglio è CUDA Warp (Column Major)
- In generale per i kernel Custom il guadagno principale avviene tra S=4096 e S=8192;
  - oltre le prestazioni tendono a stabilizzarsi

FASE 2 - Effetto della forma

CUDA Warp

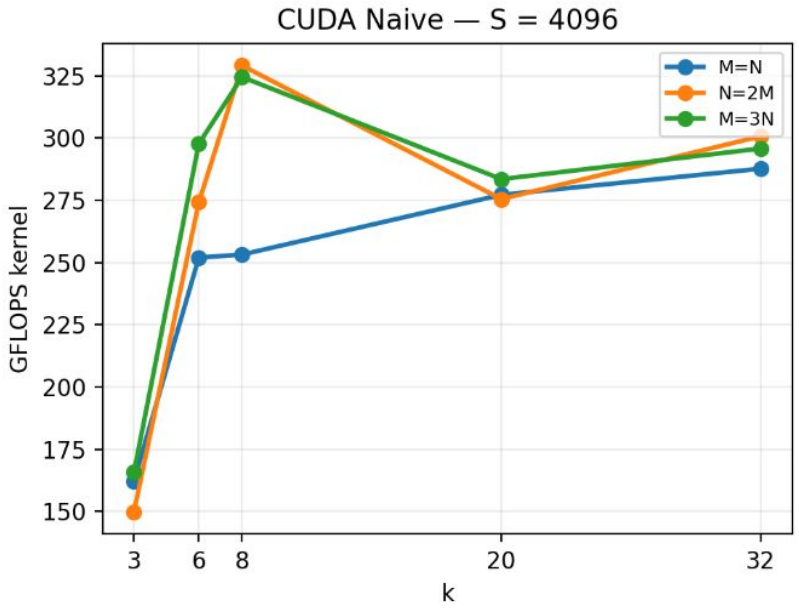


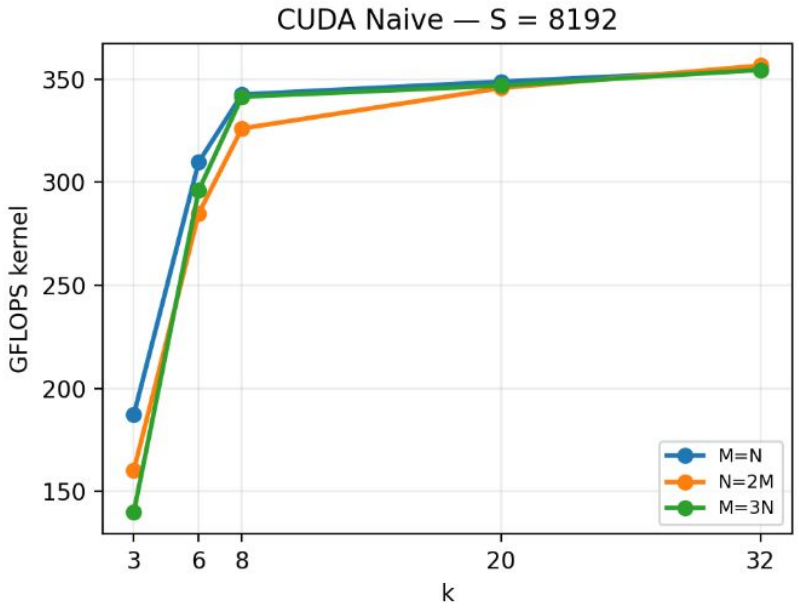


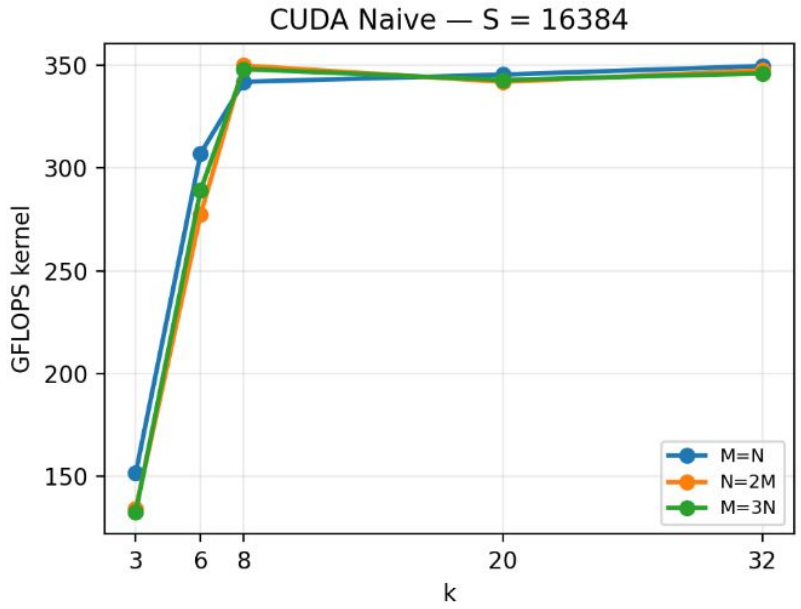


FASE 2 - Effetto della forma

CUDA Naive

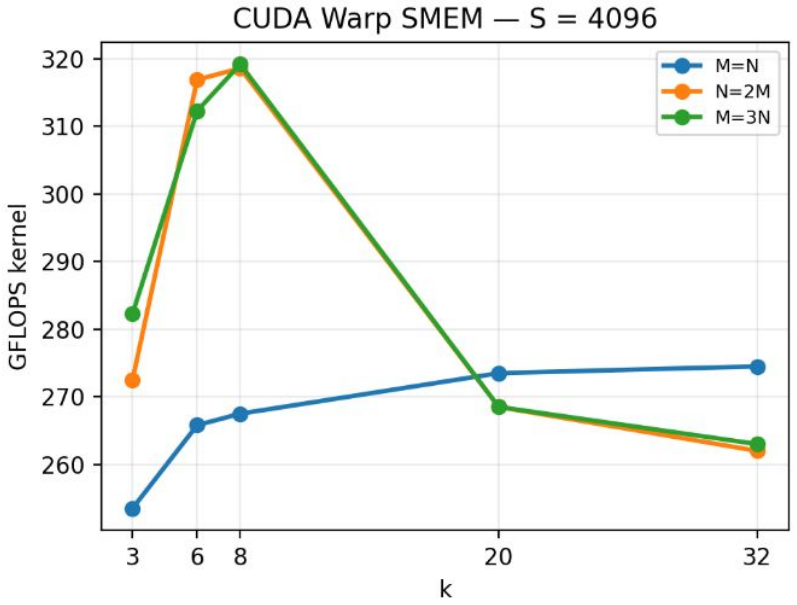


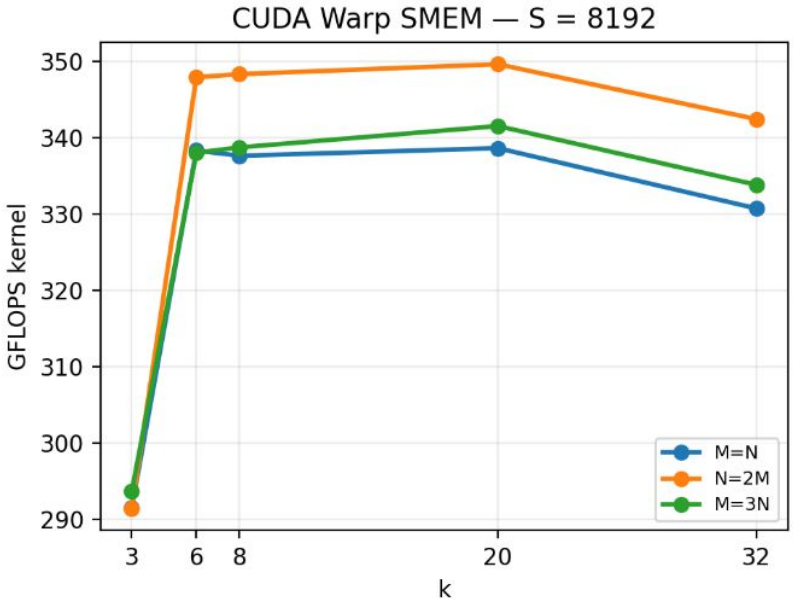


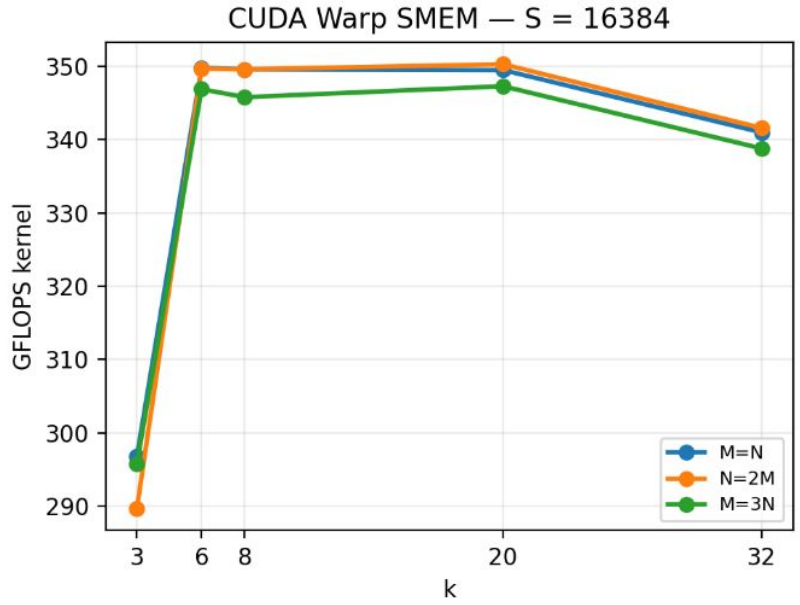


FASE 2 - Effetto della forma

CUDA Warp shared memory







FASE 2 - Effetto della forma conclusioni

Non emerge una geometria universalmente migliore.

- L’effetto della forma dipende sia dal backend sia da k.
  - Aumentando la scala i due kernel basati sulla mappatura Warp diventano quasi insensibili alla geometria.
  - CUDA Naive conserva una maggiore sensibilità, soprattutto per k piccolo

TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

FASE 3 - Effetto della precisione numerica

FASE 3 - Effetto della precisione numerica

Studio dell'effetto della precisione numerica sulle prestazioni dei quattro backend GPU.

Problema fissato:

- M=N=16384
- P=1
- Configurazioni consolidate nella fase di tuning
- Per quantificare il beneficio del cambio di precisione si utilizza lo speedup:

$$S_{32/64} = \frac{\bar{t}^{\mathrm{FP32}}_{\mathrm{kernel}}}{\bar{t}^{\mathrm{FP64}}_{\mathrm{kernel}}}$$

FASE 3 - Effetto della precisione numerica

Backend	$k = 3$	$k = 6$	$k = 8$	$k = 20$	$k = 32$
GFLOPS in FP32					
`cuda_naive`	198,3	386,8	522,0	791,6	1107,4
`cuda_warp`	604,4	1169,8	1414,3	1489,3	1945,9
`cuda_warp_smem`	601,2	1164,7	1457,2	1726,9	1473,0
`cublas`	426,7	861,1	1141,0	2962,3	4791,9
GFLOPS in FP64					
`cuda_naive`	151,5	307,1	342,0	345,5	349,7
`cuda_warp`	302,0	354,3	353,2	354,6	354,9
`cuda_warp_smem`	296,8	349,8	349,6	349,5	341,0
`cublas`	15,3	30,7	40,8	102,0	163,2
$S_{32/64}$					
`cuda_naive`	1,31	1,26	1,53	2,29	3,17
`cuda_warp`	2,00	3,30	4,00	4,20	5,48
`cuda_warp_smem`	2,03	3,33	4,17	4,94	4,32
`cublas`	27,89	28,08	27,97	29,04	29,37

FASE 3 - Effetto della precisione numerica

- Il primo risultato della campagna è che il beneficio del passaggio a FP32 non è costante. *Lo speedup dipende sia dal backend sia dalla larghezza k del multivettore.*
- Il confronto fra i tre kernel sviluppati nel progetto mostra che il cambio di precisione modifica anche i rapporti prestazionali osservati in FP64:
  - Per k ≥ 6, cuda_warp e cuda_warp_smem risultavano molto vicini. In FP32 il comportamento **si differenzia maggiormente.**
    - Per k = 3, 6, 8 i due kernel `cuda_warp` e `cuda_warp_smem` rimangono praticamente **indistinguibili**.
    - Per k = 20 il comportamento invece **cambia** → `cuda_warp_smem` ha un vantaggio di circa il 13%
    - A k = 32 il comportamento si **ribalta** completamente, con uno svantaggio del 24% rispetto
      - pressione sui registri, quantità di shared memory richiesta, sincronizzazioni, risorse necessarie per blocco, ecc.

FASE 3 - Effetto della precisione numerica

- Il backend cuBLAS presenta un comportamento nettamente diverso:
  - La libreria passa da prestazioni sensibilmente inferiori ai kernel custom in FP64 a prestazioni superiori in FP32 per i valori maggiori di k.

Nel complesso, i risultati mostrano che la scelta fra FP32 e FP64 non produce un semplice fattore di accelerazione uniforme e la precisione modifica in modo significativo il *comportamento relativo* delle diverse implementazioni.



TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

FASE 4 - Esecuzione distribuita

FASE 4 - Esecuzione distribuita

Il punto di partenza di questa fase sperimentale è un **vincolo hardware**

→ il nodo ha **una sola GPU** che *tutti* i rank utilizzano

→ aumentare P non aggiunge **capacità di calcolo**, ma aggiunge **contesa** fra contesti CUDA che, senza *Multi-Process Service (MPS)*, si devono alternare sulla scheda.

L'analisi precede in tre passi:

- Verifica della metrica lecita da utilizzare per la trattazione
- Studio dell'**effetto del numero** di rank P
- Studio dell'**effetto della forma** della griglia Pr × Pc a parità di P

FASE 4 - Esecuzione distribuita

Verifica della metrica lecita da utilizzare per la trattazione

S	P	griglia	k	`gflops`	`gflops_local_group`	rapporto	CV gruppo
4096	8	2×4	3	919,2	78,4	11,73	0,38%
8192	8	2×4	3	1025,6	172,4	5,95	0,26%
16384	8	1×8	3	465,9	250,5	1,86	0,11%
16384	8	2×4	32	296,1	298,5	0,99	0,44%

Confronto fra throughput derivato dai tempi rank-local e throughput della fase locale collettiva per casi rappresentativi di `cuda_warp`.

FASE 4 - Esecuzione distribuita

Verifica della metrica lecita da utilizzare per la trattazione

Per il vincolo hardware, la metrica che ha utilizzare è il `gflops_local_group` che è basato sul **tempo necessario affinché *tutti* i rank completino la fase locale.**

In questo modo la metrica tiene conto del lavoro effettivamente sostenuto dal device senza sovrastimarlo, come la tabella di esempio ha mostrato con i numeri.

FASE 4 - Esecuzione distribuita

Verifica della metrica lecita da utilizzare per la trattazione

- La discrepanza tra le due metriche (rank-local e local_group) non è rumore statistico come evidenziato dal *coefficiente di variazione (CV)*
- Esistono dei casi in cui due letture convergono quando ogni invocazione è abbastanza lunga.
  - Per il caso riportato come esempio, le letture convergono per k = 32
    - Con un'operazioni più piccole (k piccolo) i costi di gestione dei vari rank (scheduling, alternanza dei contesti e sincronizzazione) sono importanti rispetto al tempo di calcolo utile.

      Aumentando k, il lavoro utile domina maggiormente questi effetti.

FASE 4 - Esecuzione distribuita

Effetto del numero di rank P

Il numero di rank e la forma della griglia sono due fattori distinti.

Isoliamo l'effetto dell'aumento di P (fino a P=8), fissando la famiglia di griglie utilizzate all'interno confronti:

- P=1 → 1x1,
- P=2 → 2x1,
- P=4 → 2x2,
- P=8 → 4x2

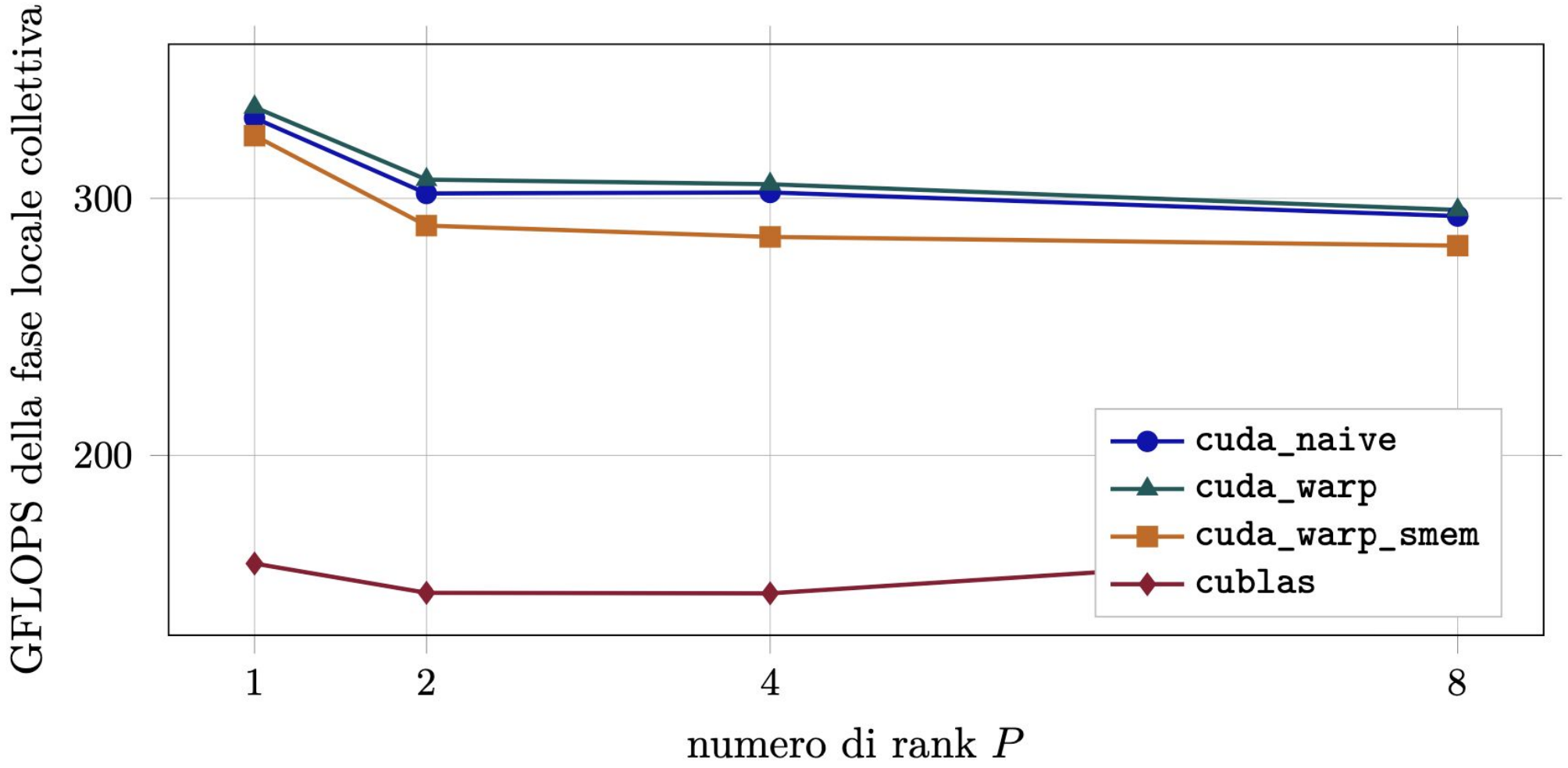
FASE 4 - Esecuzione distribuita

- *GFLOPS della fase locale collettiva*

Backend	P	$S = 4096$		$S = 8192$		$S = 16\,384$	
		$k = 3$	$k = 32$	$k = 3$	$k = 32$	$k = 3$	$k = 32$
`cuda_naive`	1	145,1	250,7	175,7	327,1	146,6	331,3
	2	97,6	270,0	174,1	290,1	166,7	301,9
	4	93,8	288,1	161,8	296,3	166,4	302,3
	8	56,0	240,3	114,8	305,0	176,4	293,1
`cuda_warp`	1	235,2	253,6	273,5	326,1	286,9	335,5
	2	160,8	275,9	243,3	292,5	247,9	307,3
	4	118,1	270,8	217,0	291,1	273,0	305,5
	8	78,2	227,5	166,7	295,5	251,9	295,5
`cuda_warp_smem`	1	232,8	236,3	267,8	304,1	277,3	324,4
	2	156,7	245,8	239,7	278,3	251,0	289,4
	4	114,5	240,5	213,9	279,7	253,2	285,0
	8	75,8	214,9	164,8	262,8	249,1	281,6
`cublas`	1	16,8	166,0	15,1	156,3	15,0	157,9
	2	13,2	134,5	15,7	163,0	13,9	146,5
	4	13,2	137,7	15,8	164,0	13,9	146,3
	8	14,4	148,6	13,2	137,0	15,6	163,5

FASE 4 - Esecuzione distribuita

Con k=32



FASE 4 - Esecuzione distribuita

Effetto del numero di rank P

- **Non compare strong scaling, e non potrebbe comparire:** il lavoro globale è invariato e la GPU è una sola, un aumento proporzionale a P sarebbe *impossibile.*
- Quello che conta non è P, ma la quantità di lavoro di ciascun rank:
  - A parità di P, per esempio fissando P = 8, si osserva come si producano effetti completamenti diversi a seconda della scala del problema
- Esistono configurazioni in cui il dividere il problema **conviene** davvero
  - Potrebbe anche dipendere dalla forma più avvantaggiata per lo specifico kernel
- Esistono configurazioni in cui la decomposizione produce un **throughput collettivo superiore** al caso a rank singolo

FASE 4 - Esecuzione distribuita

Effetto della forma della griglia di processi

Il numero di rank e la forma della griglia sono due fattori distinti.

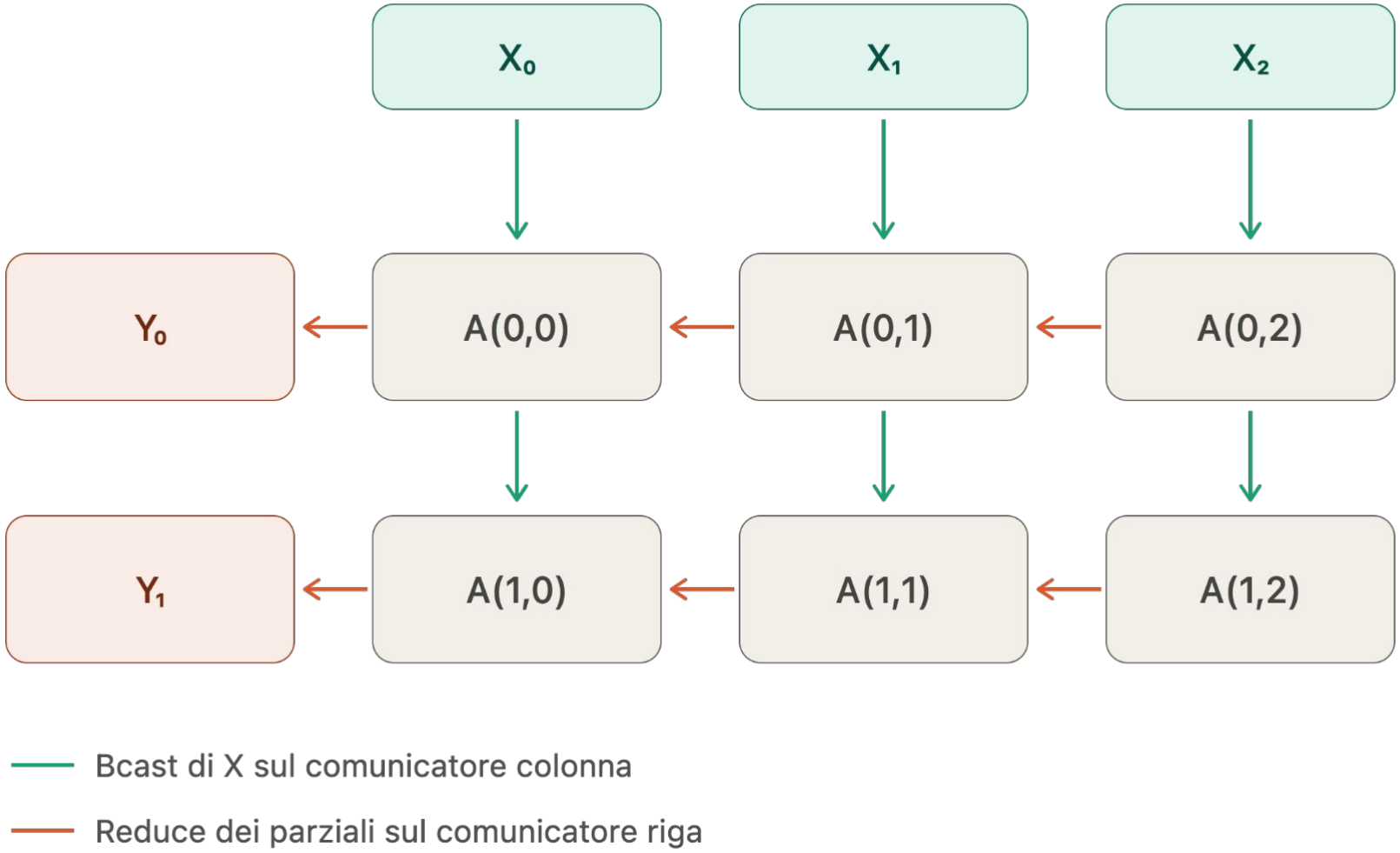
Isoliamo l'effetto dei cambiamenti della forma della griglia, fissando P=8.

A parità di P la griglia Pr × Pc decide due cose contemporaneamente:

- *Qual è la collettiva con il costo maggiore:*
  - Il *broadcast* di X avviene lungo le colonne della griglia costo → cresce con Pr
  - La *reduce* dei contributi parziali di Y avviene lungo le righe → cresce con Pc

- *Qual è la forma del blocco locale di ciascun processo*

Richiamo: Griglia & Collettive



FASE 4 - Esecuzione distribuita

Backend	$P_r \times P_c$	$k = 3$	$k = 6$	$k = 8$	$k = 20$	$k = 32$
`cuda_naive`	1×8	177,9	286,8	307,7	303,9	296,7
	2×4	168,5	279,6	309,8	305,3	302,6
	4×2	176,4	272,9	281,1	285,9	293,1
	8×1	140,5	251,5	279,9	277,1	288,8
`cuda_warp`	1×8	250,5	291,0	295,4	280,0	284,1
	2×4	254,3	305,7	310,6	303,5	298,5
	4×2	251,9	304,6	309,8	302,1	295,5
	8×1	239,8	303,0	307,4	290,7	293,7
`cuda_warp_smem`	1×8	249,4	288,2	292,7	276,2	273,3
	2×4	249,2	293,2	297,9	285,2	277,2
	4×2	249,1	297,3	303,6	296,8	281,6
	8×1	233,6	272,1	276,8	255,9	256,9
`cublas`	1×8	13,9	27,7	36,9	91,8	146,3
	2×4	13,9	27,7	36,9	91,8	146,2
	4×2	15,6	31,2	41,6	103,1	163,5
	8×1	13,1	26,2	34,8	86,3	136,8

FASE 4 - Esecuzione distribuita

- *Costo temporale complessivo delle collettive di broadcast e reduce in millisecondo*

Backend	$P_r \times P_c$	$k = 3$	$k = 6$	$k = 8$	$k = 20$	$k = 32$
`cuda_naive`	1×8	0,392	0,916	1,278	6,558	8,876
	2×4	0,149	0,378	0,438	1,616	2,594
	4×2	0,106	0,231	0,293	1,271	2,087
	8×1	0,218	0,440	0,601	1,132	1,625
`cuda_warp`	1×8	0,373	0,867	1,317	5,937	8,636
	2×4	0,146	0,386	0,429	1,657	2,428
	4×2	0,101	0,209	0,302	1,282	2,190
	8×1	0,219	0,438	0,604	1,966	3,438
`cuda_warp_smem`	1×8	0,392	0,889	1,265	6,671	8,653
	2×4	0,145	0,378	0,439	1,702	2,580
	4×2	0,108	0,219	0,291	1,261	1,949
	8×1	0,232	0,448	0,602	1,897	1,749
`cublas`	1×8	0,439	0,932	1,286	6,123	8,673
	2×4	0,173	0,405	0,493	1,621	2,725
	4×2	0,122	0,238	0,325	1,351	1,997
	8×1	0,239	0,497	0,658	1,975	1,739

FASE 4 - Esecuzione distribuita

Per k=32

Backend	$P_r \times P_c$	t_{bcast}	t_{kernel}	t_{reduce}	T	$t_{\text{local}}^{\text{gruppo}}$	GFLOPS	GFLOPS gruppo
`cuda_naive`	1×8	0,000	55,010	8,875	63,250	57,906	271,6	296,7
	2×4	0,374	54,995	2,220	57,385	56,776	299,4	302,6
	4×2	1,506	56,318	0,581	57,786	58,617	297,3	293,1
	8×1	1,408	55,719	0,218	57,322	59,487	299,7	288,8
`cuda_warp`	1×8	0,000	58,763	8,635	66,000	60,481	260,3	284,1
	2×4	0,342	55,736	2,085	58,022	57,555	296,1	298,5
	4×2	1,586	55,881	0,604	57,453	58,145	299,0	295,5
	8×1	3,227	55,049	0,211	58,443	58,496	294,0	293,7
`cuda_warp_smem`	1×8	0,000	61,158	8,652	67,886	62,850	253,1	273,3
	2×4	0,379	60,132	2,201	61,918	61,974	277,5	277,2
	4×2	1,356	58,684	0,593	59,954	61,000	286,6	281,6
	8×1	1,543	63,288	0,206	65,012	66,868	264,3	256,9
`cublas`	1×8	0,001	115,798	8,672	121,719	117,440	141,1	146,3
	2×4	0,406	115,790	2,319	118,181	117,521	145,4	146,2
	4×2	1,401	102,778	0,595	104,035	105,093	165,1	163,5
	8×1	1,499	121,909	0,240	123,607	125,604	139,0	136,8

FASE 4 - Esecuzione distribuita

Effetto della forma della griglia di processi

- La forma della griglia sposta effettivamente il costo fra le due collettive, come ci si aspetterebbe
- Le due collettive hanno effettivamente costo differente, come ci si aspetterebbe
- A parità di P, il lavoro locale rimane pressoché costante ma cambia la sua geometria
- Il costo delle collettive cresce con k, ma la crescita dipende fortemente dalla forma della griglia

  Per i valori piccoli di k il costo delle collettive rimane molto contenuto

  - Per esempio, la griglia 1 × 8, che concentra la comunicazione sulla *reduce*, diventa progressivamente più costosa all'aumentare di k.

FASE 4 - Esecuzione distribuita

Effetto della forma della griglia di processi

- La griglia che massimizza il throughput dipende dal backend, ma tende a rimanere stabile al variare di k
  - cuda_warp con forma della griglia pari a 2x4 fornisce `gflops_local_group` maggiore per tutti i valori di k considerati
- La sensibilità alla geometria del blocco locale è maggiore per cuBLAS:
  - cuBLAS raggiunge il taglio maggiore nel valore `gflops_local_group` tra differenti griglie → circa il 20%
  - Per i kernel sviluppati tale effetto è presente ma, nel punto considerato, risulta generalmente più contenuto → intorno al 5% e circa il 10% al massimo (per `cuda_warp_smem`)

TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

GRAZIE DELL'ATTENZIONE